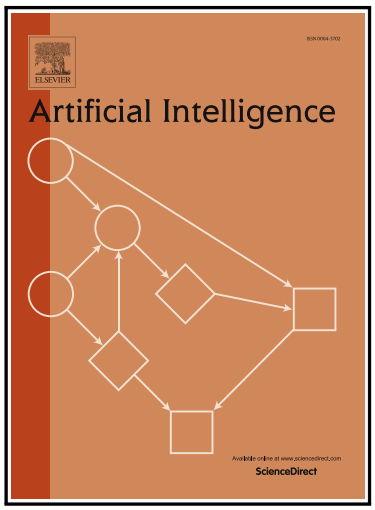

Similarity of Planning Domain Models via Answer Set Programming

Lukáš Chrpa, Carmine Dodaro, Marco Maratea, Marco Mochi, Mauro Vallati

PII: S0004-3702(26)00146-3
DOI: https://doi.org/10.1016/j.artint.2026.104620
Reference: ARTINT 104620

To appear in: *Artificial Intelligence*

Received date: 25 July 2024
Revised date: 26 August 2026
Accepted date: 31 August 2026

Please cite this article as: Lukáš Chrpa, Carmine Dodaro, Marco Maratea, Marco Mochi, Mauro Vallati, Similarity of Planning Domain Models via Answer Set Programming, *Artificial Intelligence* (2025), doi: https://doi.org/10.1016/j.artint.2026.104620

Similarity of Planning Domain Models via Answer Set Programming

Lukáš Chrpa[a], Carmine Dodaro[b], Marco Maratea[b,*], Marco Mochi[c], Mauro Vallati[d]

[a]*Czech Technical University in Prague, Prague, Czech Republic,*
[b]*University of Calabria, Rende, Italy,*
[c]*University of Genova, Genova, Italy,*
[d]*University of Huddersfield, Huddersfield, UK,*

Abstract

Automated planning is a prominent area of Artificial Intelligence, and an important component for intelligent autonomous agents. A critical aspect of domain-independent planning is the domain model, that encodes a formal representation of domain knowledge needed to reason about a given problem. Despite the crucial role of domain models in automated planning, there is a lack of tools supporting the knowledge engineering process by comparing different versions of the models, in particular, determining and highlighting differences the models have.

In this paper, we formalise the notions of *structural equivalence* and *strong equivalence* of domain models, and, on top of these, we introduce a novel concept of *similarity* of domain models. To measure the similarity of two models, we introduce a directed graph representation of lifted domain models that allows the formulation of the domain model similarity problem as a variant of the graph edit distance problem. We propose an Answer Set Programming approach to optimally solve the domain model similarity problem, that identifies the minimum number of modifications the models need to become strongly equivalent, and we demonstrate the capabilities of the approach on a range of benchmark models.

Keywords: Automated Planning, Answer Set Programming, Domain Model Similarity

1. Introduction

Automated planning is a research discipline that addresses the problem of generating a totally- or partially-ordered sequence of actions that transforms the environment from an initial state to a desired goal state. It has matured to such a degree that there exists a wide range of applications utilising planning, including UAV manoeuvring (Ramírez et al., 2018), space exploration (Ai-Chang et al., 2004), and train dispatching (Cardellini et al., 2021).

A critical aspect of domain-independent planning is the domain knowledge that must be fed into a planning engine that comes under the form of a domain model, a symbolic

*Corresponding author

Email addresses: chrpaluk@cvut.cz (Lukáš Chrpa), carmine.dodaro@unical.it (Carmine Dodaro), marco.maratea@unical.it (Marco Maratea), marco.mochi@edu.unige.it (Marco Mochi), m.vallati@hud.ac.uk (Mauro Vallati)

This paper is an extended and revised version of a conference paper that appeared in the proceedings of the 2023 European Conference on Logics in Artificial Intelligence (JELIA) (Chrpa et al., 2023).

representation of the environment and actions, that has to be engineered prior to its use (McCluskey and Porteous, 1997). The importance of good quality domain models in planning, and of the corresponding knowledge engineering process, has been well-argued (McCluskey et al., 2017; Vallati and McCluskey, 2021; Vallati and Chrpa, 2019). However, there is a lack of approaches to support the knowledge engineering process, and with the widespread use of LLM-based techniques, there is the concrete risk of generating a large number of low quality models (Oswald et al., 2024). In particular, there is no "diff" tool that compares different versions of a domain model and highlights differences among them. Tools such as D-VAL (Shrinah et al., 2021) or the more recent work of Coulter et al. (2022) provide some limited support to compare domain models focusing on the state space they can generate, and the model reconciliation problem focuses on explaining why two models cannot create the same optimal plans (Chakraborti et al., 2017).

To address the highlighted research gap, we propose a novel concept of domain model similarity and present a theoretical framework underlying the concept, which employs an extension of the notion of *strong equivalence* informally introduced by Shoeeb and McCluskey (2011), which determines whether domain models are the same except for naming. Besides formalising the notion of *strong equivalence* of domain models, we introduce and formalise the notion of *structural equivalence* (of domain models) as a necessary condition (but not necessarily sufficient condition) for strong equivalence. We start with domain models expressed in classical STRIPS planning, and then extend the framework by introducing negative preconditions and durative actions as introduced in PDDL2.1 (Fox and Long, 2003). We propose a directed graph representation of lifted domain models and we show that domain models are structurally equivalent if and only if the graphs representing them are isomorphic. For strong equivalence, on top of structural equivalence, the numeric attributes, such as action durations or action costs, must be equal. Then, we define the *domain model similarity problem* between domain models as the minimum number of modifications that have to be made to both models to make them strongly equivalent. It corresponds to a variant of the well-known notion of *edit distance* between two graphs (representing the domain models). The introduced theoretical framework gives us the notion of *similarity* by measuring the distance between domain models and enumerating the modifications that need to be done to make the models (strongly) equivalent. Then, we present an approach based on Answer Set Programming (ASP) (Baral, 2003; Gelfond and Lifschitz, 1991; Niemelä, 1999; Brewka et al., 2011) that allows comparing planning domain models to assess their similarity. Our solution relies on a directed graph representation of the lifted domain models, and is capable of solving the domain model similarity problem. Beside providing the first concrete approach to assess if two domain models are strongly equivalent, the proposed notion of similarity, and the ASP-based approach to measure it, have several practical implications: (i) it can be incorporated into a "diff" tool for highlighting differences between two versions of a domain model, to help knowledge engineers in understanding modifications; (ii) it can support the evaluation of tools for automated domain model acquisition (e.g., LOCM (Cresswell et al., 2013)) by comparing acquired domain models to the reference domain models; (iii) it can be exploited as an advanced plagiarism checker, where it can provide a "similarity" score to flag potential cases of plagiarism; (iv) it can support the evaluation of models in competitions on domain modelling such as the International Competition on Knowledge Engineering for Planning and Scheduling (ICKEPS) (Chrpa et al., 2017); and (v) it can help to compute "common cores" to support inheritance and modularity for modelling PDDL planning models (Lindsay, 2023).

We evaluate the approach on well-known benchmark domains from international com-

petitions involving classical planning domains and PDDL2.1 domains containing durative actions, of different sizes with regard to the number of models' predicates and operators, on ICKEPS domains (point (iv) above), and on the problem of computing "common cores" of planning domains (point (v) above). Our fully declarative approach is able to compare a number of planning domains, except the largest, and an improved solution that exploits a pre-processor via imperative programming that acts as a sort of "problem-aware pre-grounder", which complements the declarative encoding, is able to compare all planning domains analyzed. The related empirical evaluation shows that, by employing the improved solution, the comparison can be performed in less than a CPU-time second for the planning models coming from IPC competitions, hence suggesting that it can be fruitfully exploited to support the knowledge engineering process of domain models in real time. This paper is an extended and revised version of (Chrpa et al., 2023), whose main additions are: (i) We extended the whole framework, which in the JELIA 2023 paper considered classical STRIPS domains only, to include negative preconditions and some PDDL2.1 elements, i.e., durative actions and action cost. (ii) We extended the experimental analysis, other than using PDDL2.1 domains, to include the computation of "common structures" of planning domains (Section 7.4). (iii) We added a proposition with corresponding proof about the correctness of the proposed ASP program for solving the introduced problem. (iv) We significantly expanded the related work (next section), and added more explanation, motivation and justification.

The remainder of this paper is structured as follows. In Section 2 we discuss related literature. In Section 3, we present needed preliminaries about automated planning, graph similarity and ASP. Then, Section 4 reviews and extends the concept of domain model equivalence, while Section 5 presents a novel concept of domain model similarity with a related problem, respectively. Section 6 presents our approach based on ASP, while Section 7 is devoted to the experimental analysis. The paper ends in Section 8 with a conclusion.

2. Related Work

The general notion of quality of planning domain models and of the corresponding knowledge engineering processes has been widely investigated by the research community. While the knowledge engineering in the planning and scheduling field was formally defined in 2003 (Biundo et al., 2003), significant effort was already dedicated to support the generation of models for domain-independent planning systems. For example, Aitken and Shadbolt (1994) proposed an approach based on KADS, a proven methodology for knowledge base systems development, to structure the engineering of planning models, and McCluskey and Porteous (1997) introduced tools and methods for promoting the quality of the resulting planning models. McCluskey et al. (2017) introduced a number of general properties that can help in understanding the quality of a domain model, while Shah et al. (2013) proposed an application-specific investigation of how different knowledge engineering processes can affect planning models. Quantitative metrics to manually compare models were introduced as part of the 2016 edition of ICKEPS (Chrpa et al., 2017). Vallati and McCluskey (2021) proposed a quality framework for assessing models and comparing engineering processes and tools. To support the engineering of models in planning, notions such as design patterns and inheritance have been borrowed from well-established areas of software engineering (Lindsay, 2023; Simpson et al., 2002; Vallati and McCluskey, 2020). Differently from such works, our paper introduce a quantitative metric for comparing in an automated way two models of planning domains.

Significant effort has been dedicated to the design of approaches to generate and maintain domain models for planning. Work in the area includes: (i) frameworks for use by knowledge engineers, such as GIPO (Simpson et al., 2007), itSIMPLE (Vaquero et al., 2007), KEWI (Wickler et al., 2014), and, to some extent, the planning.domains system[2]; (ii) approaches to automated or semi-automated domain model acquisition from observations (Cresswell et al., 2013; Mourão et al., 2012; Gregory and Lindsay, 2016; Arora et al., 2018; Mordoch et al., 2024; Aineto and Scala, 2024) or Large Language Models (Oswald et al., 2024); (iii) techniques to assist in refining and extending existing models (Lindsay et al., 2020; Porteous et al., 2021); and (iv) techniques to reformulate existing models to improve operationality and solvability of models (Chrpa and Vallati, 2022; Alarnaouti et al., 2023). Our work, instead, focuses on the comparison of models.

The notion of strong domain model equivalence has been informally introduced in Shoeeb and McCluskey (2011). Leveraging on a similar notion, Shrinah et al. (2021) proposed an automated approach, D-VAL, to check the functional equivalence of two domain models, i.e., their ability to parse the same set of problems. D-VAL focuses on comparing models of the same domain that have been reformulated to ensure that the reformulation process did not undermine the domain model capabilities. The approach proposed in this paper is more general and allows to compare even very different models in terms of their corresponding solution spaces and to obtain a measure of their similarity. Coulter et al. (2022) introduced an approach to compare two planning tasks based on their ability to reach exactly the same state space (from the initial states of both tasks). This approach assumes that the compared models (of the planning tasks) share part of the structure in terms of objects, types, and operator signatures. In contrast, our work compares domain models and thus is not planning task specific.

The model reconciliation problem (MRP) by (Chakraborti et al., 2017; Sreedharan et al., 2021) focuses on identifying and fixing the reasons why two domain models are not able to generate the same optimal plans. It is worth noting that the reconciled models are by no means functionally equivalent as the process is performed with regards to an individual planning problem. Further, the reconciliation approach requires the models to share predicates and operators. Sreedharan et al. (2019) extended the MRP by interacting with a human user to explain some transitions in a considered model. The work still focuses on individual problems, and, differently from our approach, does not allow to compare the overall planning models considered. A different line of work focused on the MRP is concerned with identifying the planning domain model that better explains a sequence of observations (Aineto et al., 2019). In a related fashion, the model characterisation problem has been recently introduced to help define the characteristics of planning domain models to support formal classification (Grundke et al., 2024). Thus, MRP focuses on optimal plans and on single problem instances, while our approach is more general.

Notably, an approach based on ASP has also been proposed for the MRP in planning (Nguyen et al., 2020), while Son et al. (2021) deals with the MRP but is specifically defined on two logic programs and their answer sets. Some authors of Son et al. (2021) followed a similar direction in (Nguyen et al., 2021), but in the context of (numerical) scheduling and employing the CLINGO-DL language, which is an extension of ASP enriched with a limited form of arithmetic (Janhunen et al., 2017).

[2] planning.domains

3. Background

In this section, we present, in separate subsections, the needed preliminaries about automated planning, graph similarity and answer set programming, respectively.

3.1. Automated Planning

Traditionally, automated planning deals with finding a sequence of actions transforming the environment from some initial state to a desired goal state (Ghallab et al., 2004).

In the *grounded STRIPS representation*, the environment is represented by *propositions* (or *facts*), where each proposition represents an element of the environment we want to observe (e.g., package "x" is inside truck "t"). *States* are defined as sets of these propositions such that if a proposition is in the set it is considered true in that state, otherwise it is considered false. An *action* is a quadruple $a = (name(a), pre(a), del(a), add(a))$, where $name(a)$ represents a unique action name, $pre(a)$, $del(a)$ and $add(a)$ are sets of propositions representing the *precondition* of a, the *delete* and *add effects* of a, respectively. We assume a is always *well defined*, i.e., $del(a) \cup add(a) \neq \emptyset$ (as an action without any effect would be useless). We say that an action a is *applicable* in a state s if and only if $pre(a) \subseteq s$. Application of a in s (if possible) results in a state $(s \setminus del(a)) \cup add(a)$.

A *(grounded) domain model* $\mathcal{D} = (L, A)$ is specified via a set of propositions (facts) L and a set of actions A. A *problem instance* $\mathcal{P} = (I, G)$ for $\mathcal{D}$ is specified via the initial state $I \subseteq L$ and a set of propositions representing the goal $G \subseteq L$.

In the *lifted STRIPS representation*, the environment is represented by first-order logic *predicates* that are specified via unique predicate symbols (representing their names) and lists of terms, which are free variables or constants. Predicates capture general relations between objects (e.g., a truck is at a location). A state of the environment is specified via a set of grounded predicates (all free variables are substituted by constants), where if a (grounded) predicate is in the set it is considered true in that state, otherwise it is considered false. A *planning operator* $o = (name(o), pre(o), del(o), add(o))$ is specified such that $name(o) = op\_name(x_1, \ldots, x_k)$ (*op_name* represents a unique operator name and $x_1, \ldots x_k$ are variable symbols (parameters) appearing in the operator), $pre(o)$ is a set of predicates representing the operator's *preconditions*, $del(o)$ and $add(o)$ are sets of predicates representing the operator's *delete* and *add* effects, respectively. Again, we assume o is always *well defined*, i.e., $del(o) \cup add(o) \neq \emptyset$. Planning operators can be understood as templates for actions, as they can describe general mechanics (e.g., moving trucks between locations, loading packages into trucks).

A *lifted domain model* $\mathcal{D} = (P, O)$ is specified via a set of predicates P such that each predicate has a distinct predicate symbol (or name) and all terms are free variables, and a set of planning operators O such that each operator has a unique name. Lifted domain models enable a compact representation of the state space and actions of a given domain.

A *problem instance* $\mathcal{P} = (Obj, I, G)$ for a lifted domain model $\mathcal{D}$ is specified via a set of objects (constants) Obj, the initial state I and a set of grounded predicates representing the goal G. Given the set of predicates P and the set of operators O from the lifted domain model, and the set of objects Obj from the problem instance, we can generate a grounded domain model such that a set L contains all possible instances of predicates from P obtained by substituting their free variables by the constants from Obj. Note that although L in this case contains grounded predicates, they can be treated in the same way as propositions (facts). In a similar fashion, we can generate a set of actions A by considering all possible substitutions of free variables in the parameters of all operators from O by the constants from Obj.

A *planning task* $(\mathcal{D}, \mathcal{P})$ consists of a domain model $\mathcal{D}$ and a problem instance $\mathcal{P}$. A *solution plan* for a planning task is a (partially-ordered) sequence of actions such that the consecutive application of the actions in the plan (starting in the initial state) results in a state in which all the goal facts are true. For more details about (grounded and lifted) STRIPS representation, an interested reader is referred to Ghallab et al. (2004).

We say that predicates are *equal* if they have the same predicate symbols and their parameters, including their order, are identical. We define a function $sym(\cdot)$ that returns the predicate symbol (or name) of a predicate. Then, we define a function $pars(\cdot)$ that returns the set of variable symbols of a predicate or an operator. We also define a function $arity(\cdot)$ that returns the number of variable symbols of a predicate or an operator. With regard to *substitution mappings* that map free variables into terms (variables or constants in our case), we use a specific notation in order to disambiguate with other types of mappings. In particular, for a substitution mapping χ and a predicate (or an operator) $p(x_1, \ldots, x_n)$, $(p|\chi)$ refers to substituting $x_1, \ldots, x_n$ for terms according to χ, i.e., $(p|\chi) \equiv p(\chi(x_1), \ldots, \chi(x_n))$. Substitution mapping can be applied for other structures (including other substitution mappings) analogously.

3.1.1. Planning Domain Definition Language (PDDL)

PDDL is a widely used language for specifying planning tasks, decoupled into domain models and problem instances. PDDL was introduced in 1998 for the purpose of the first International Planning Competition (IPC) (Mcdermott et al., 1998). PDDL is inspired by the STRIPS planner representation, which is predicate-centric and aligns with the definition of "lifted STRIPS representation" as above (Fikes and Nilsson, 1971). In the subsequent years, PDDL has evolved to allow representing more expressive planning features. Perhaps the most prominent version of PDDL is version 2.1 (Fox and Long, 2003) that introduced durative actions (apart from other features).

To align with the planning literature, we will use the PDDL syntax to describe predicates, operators and actions in the running examples throughout the paper. In PDDL, free variables start with "?", i.e., ?x denotes a free variable named "x". Predicates are specified in form (⟨predicate-name⟩ ⟨term⟩*) and actions/operators in form ⟨operator-name⟩(⟨term⟩*). "Term" denotes either a free variable or a constant. Constants, predicate names and operator names are strings beginning with a letter and containing letters, digits, hyphens and underscores (for more details, see (Mcdermott et al., 1998)).

Example 1 (Running example)**.** *We consider as a running example a simplified version of the well-known Logistics domain, originally introduced in the IPC 2000. In the simplified version, a number of trucks are used to deliver packages from a location of origin to a destination location. The domain model includes 4 predicates:* (at-truck ?Loc ?Truck), (at-package ?Loc ?Pkg), (in-package ?Pkg ?Truck), (in-city ?Cty ?Loc) *and 3 operators:* load(?Loc ?Pkg ?Truck), unload(?Loc ?Pkg ?Truck), *and* move(?Cty ?Loc1 ?Loc2 ?Truck).

3.1.2. Beyond Classical (STRIPS) Planning

Extending the concept of a planning operator beyond STRIPS, most notably to reflect *durative actions* that were introduced in PDDL 2.1 (Fox and Long, 2003), involves defining operators that contain more sets of predicates (e.g. to reflect "at start" or "at end" preconditions and effects etc.) and numeric attributes (e.g. to reflect action duration and/or action cost). Note that we consider the semantics of PDDL 2.1 durative actions, in which preconditions and effects are specified by a relative time of their occurrence.

We define an *extended planning operator* as a tuple $o = (name(o), attr_1(o), \ldots, attr_m(o), Z^1(o), \ldots, Z^n(o))$, where $name(o) = op\_name(x_1, \ldots, x_k)$ (op_name represents a unique operator name and $x_1, \ldots x_k$ are variable symbols (parameters) appearing in the operator), $attr_1(o), \ldots, attr_m(o)$ are numeric attributes, and $Z^1(o)$, $\ldots$, $Z^n(o)$ are sets of predicates. An *extended lifted domain model* $\mathcal{D} = (P, O)$ is specified via a set of predicates P and a set of extended operators O (in analogy to a lifted domain model).

Notably, the extended planning operator definition generalizes the lifted STRIPS planning operator definition, since $pre(o), del(o), add(o)$ are sets of predicates that correspond to the $Z^1(o), Z^2(o), Z^3(o)$ sets, respectively, in the extended operator definition. A planning operator that also contain *negative preconditions* and *action cost* can also be represented by an extended planning operator, where $attr_1(o)$ corresponds to the action cost, and the $Z^1(o), Z^2(o), Z^3(o), Z^4(o)$ sets of predicates correspond to the positive and negative preconditions, and delete and add effects.

3.2. Graph Similarity

Comparing graphs, in terms of how similar they are, belongs under the umbrella of *graph matching* (Bengoetxea, 2002). For our purpose, we will consider (labelled) directed graphs with different types of edges.

Let $\mathcal{G} = (V, E^1, E^2, \ldots, E^k)$ be a *multi-edged directed graph*, where V is a set of vertices and $E^1, \ldots, E^k$ are sets of edges. We say that multi-edged directed graphs $\mathcal{G}_1 = (V_1, E_1^1, \ldots, E_1^k)$ and $\mathcal{G}_2 = (V_2, E_2^1, \ldots, E_2^k)$ are *isomorphic* if and only if there exist bijective mapping $\xi : V_1 \to V_2$ such that for each $1 \leq i \leq k : (x, y) \in E_1^i \Leftrightarrow (\xi(x), \xi(y)) \in E_2^i$.

A multi-edged directed graph $\mathcal{G} = (V, E^1, E^2, \ldots, E^k)$ is called *labelled* if there exists a set of labels L and mappings $\mathcal{L}^1 : E^1 \to L, \ldots, \mathcal{L}^k : E^k \to L$ that assign labels to each edge. Let $\mathcal{G}_1, \mathcal{G}_2$ and ξ be defined as above, and let L_1 and L_2 be sets of labels for $\mathcal{G}_1$ and $\mathcal{G}_2$, respectively, and let $\mathcal{L}_1^1, \ldots, \mathcal{L}_1^k$ and $\mathcal{L}_2^1, \ldots, \mathcal{L}_2^k$ be mappings from edges to labels defined analogously as above. If there exists $\nu : L_1 \to L_2$ such that for each $1 \leq i \leq k : (x, y) \in E_1^i \Leftrightarrow (\xi(x), \xi(y)) \in E_2^i \wedge \nu(\mathcal{L}_1^i((x, y))) \equiv \mathcal{L}_2^i((\xi(x), \xi(y)))$, then $\mathcal{G}_1$ and $\mathcal{G}_2$ are *isomorphic*.

For the sake of simplicity, we will denote labelled edges in the form (x, l, y) which is a shortcut of $\mathcal{L}^i(x, y) = l$.

Let $\mathcal{G}_1'$ and $\mathcal{G}_2'$ be subgraphs of $\mathcal{G}_1$ and $\mathcal{G}_2$, respectively. We say that $\mathcal{G}_1'$ and $\mathcal{G}_2'$ are *common isomorphic subgraphs* of $\mathcal{G}_1$ and $\mathcal{G}_2$ if and only if $\mathcal{G}_1'$ and $\mathcal{G}_2'$ are isomorphic.

Let $elem$ denote the number of elements in a graph (with k different types of edges), i.e., for $\mathcal{G} = (V, E^1, \ldots, E^k)$, $elem(\mathcal{G}) = |V| + \sum_{i=1}^{k} |E^i|$. We say that $\mathcal{G}_1'$ and $\mathcal{G}_2'$ are *maximum common isomorphic subgraphs* of $\mathcal{G}_1$ and $\mathcal{G}_2$ if and only if (i) $\mathcal{G}_1'$ and $\mathcal{G}_2'$ are common isomorphic subgraphs of $\mathcal{G}_1$ and $\mathcal{G}_2$ and (ii) for every pair $\mathcal{G}_1''$ and $\mathcal{G}_2''$ being also common isomorphic subgraphs of $\mathcal{G}_1$ and $\mathcal{G}_2$ it is the case that $elem(\mathcal{G}_1') \geq elem(\mathcal{G}_1'')$ (and $elem(\mathcal{G}_2') \geq elem(\mathcal{G}_2'')$). Then, we define a function *dist* representing a *distance* between graphs $\mathcal{G}_1$ and $\mathcal{G}_2$ as $dist(\mathcal{G}_1, \mathcal{G}_2) = elem(\mathcal{G}_1) + elem(\mathcal{G}_2) - 2 * elem(\mathcal{G}_1')$ with $\mathcal{G}_1'$ and $\mathcal{G}_2'$ being maximum common isomorphic subgraphs of $\mathcal{G}_1$ and $\mathcal{G}_2$. Note that our notion of distance is a variant of the *Graph Edit Distance* (Sanfeliu and Fu, 1983) in which vertex and edge substitutions are not explicitly counted. The notion of distance means the number of changes that need to be made to both graphs to make them isomorphic, that, in our context, refers to the number of changes to domain models (e.g. adding predicates into the model, or preconditions or effects into the actions) to make them (structurally) equivalent.

3.3. Answer Set Programming

Answer Set Programming (ASP) (Brewka et al., 2011) is a programming paradigm developed in the area of KRR and logic programming. In this section, we overview the language of ASP (Calimeri et al., 2020). Note that the ASP terminology may not be perfectly aligned to the one of planning in the usage of some terms, e.g., atoms and predicates.

3.3.1. Syntax

The syntax of ASP is similar to the one of Prolog. Variables are strings starting with an uppercase letter, and constants are integers or strings starting with lowercase letters. A *term* is either a variable or a constant. A *standard atom* is an expression `p(`t_1`,...,`t_n`)`, where `p` is a *predicate* of arity n and $t_1, \ldots, t_n$ are terms. An atom `p(`t_1`,...,`t_n`)` is ground if $t_1, \ldots, t_n$ are constants. A *ground set* is a set of pairs of the form $\langle consts : conj \rangle$, where $consts$ is a list of constants and $conj$ is a conjunction of ground standard atoms. A *symbolic set* is a set specified syntactically as `{Terms`$_1$` : Conj`$_1$`; ..., Terms`$_t$` : Conj`$_t$`}` where $t > 0$, and for all $i \in [1, t]$, each `Terms`$_i$ is a non-empty list of terms, and each `Conj`$_i$ is a non-empty conjunction of standard atoms. A *set term* is either a symbolic set or a ground set. Intuitively, a set term `{X:a(X,c),p(X); Y:b(Y,m)}` stands for the union of two sets: the first one contains the `X`-values making the conjunction `a(X,c), p(X)` true, and the second one contains the `Y`-values making the conjunction `b(Y,m)` true. An *aggregate function* is of the form $f(S)$, where S is a set term, and f is an *aggregate function symbol*. Basically, aggregate functions map multisets of constants to a constant. The most common functions implemented in ASP systems are `#count`, for counting number of terms; `#sum`, for computing sum of integers, `#min`, for computing the minimum integer in a set, and `#max`, for computing the maximum integer in a set (Faber et al., 2011). An *aggregate atom* is of the form $f(S) \prec T$, where $f(S)$ is an aggregate function, $\prec \in \{$`<, <=, >, >=, !=, =`$\}$ is a comparison operator, and T is a term called guard. An aggregate atom $f(S) \prec T$ is ground if T is a constant and S is a ground set. An *atom* is either a standard atom or an aggregate atom. A *rule* r has the following form:

`a`$_1$ `| ... |` `a`$_n$ `:-` `b`$_1$`, ...,` `b`$_k$`, not b`$_{k+1}$`, ..., not b`$_m$`.`

where `a`$_1$`,...,a`$_n$ are standard atoms (with $n \geq 0$), `b`$_1$`,...,b`$_k$ are atoms, and `b`$_{k+1}$`,...,b`$_m$ are standard atoms (with $m \geq k \geq 0$). A literal is either a standard atom `a` or its negation `not a`. The disjunction `a`$_1$ `| ... |` `a`$_n$ is the *head* of r, while the conjunction `b`$_1$`, ..., b`$_k$`, not b`$_{k+1}$`, ..., not b`$_m$ is its *body*. Rules with empty body are called *facts*. Rules with empty head are called *constraints*. A variable that appears uniquely in set terms of a rule r is said to be *local* in r, otherwise it is a *global* variable of r. An ASP program is a set of *safe* rules, where a rule r is *safe* if the following conditions hold: *(i)* for each global variable X of r there is a positive standard atom ℓ in the body of r such that X appears in ℓ; and *(ii)* each local variable of r appearing in a symbolic set `{Terms : Conj}` also appears in a positive atom in `Conj`. A *weak constraint* ω (Buccafurri et al., 2000) is of the form:

`:~ b`$_1$`, ...,` `b`$_k$`, not b`$_{k+1}$`, ..., not b`$_m$`. [w@l,` t_1`, ...,` t_z`]`

where $t_1, \ldots, t_z$ are terms, `w` and `l` are the weight and level of ω, respectively. Intuitively, `[w@l]` is read "as weight `w` at level `l`", where weight is the "cost" of violating the condition in the body of `w`, whereas levels can be specified for defining a priority among preference criteria. An ASP program with weak constraints is $\Pi = \langle P, W \rangle$, where P is a program and W is a set of weak constraints. A standard atom, a literal, a rule, a program or a weak constraint is *ground* if no variables appear in it.

3.3.2. Semantics

Let P be an ASP program. The *Herbrand universe* U_P and the *Herbrand base* B_P of P are defined as usual. The ground instantiation G_P of P is the set of all the ground instances of rules of P that can be obtained by substituting variables with constants from U_P. An *interpretation* I for P is a subset I of B_P. A ground standard atom `p` is true w.r.t. I if `p` $\in I$, and false otherwise. A literal `not p` is true w.r.t. I if `p` is false w.r.t. I, and false otherwise. An aggregate atom is true w.r.t. I if the evaluation of its aggregate function (i.e., the result of the application of f on the multiset S) with respect to I satisfies the guard; otherwise, it is false. A ground rule r is *satisfied* by I if at least one atom in the head is true w.r.t. I whenever all conjuncts of the body of r are true w.r.t. I. A model is an interpretation that satisfies all rules of a program. Given a ground program G_P and an interpretation I, the *reduct* of G_P w.r.t. I is the subset G_P^I of G_P obtained by deleting from G_P the rules in which a body literal is false w.r.t. I (Faber et al., 2011). An interpretation I for P is an *answer set* (or stable model) for P if I is a minimal model (under subset inclusion) of G_P^I (i.e., I is a minimal model for G_P^I). Given a program with weak constraints $\Pi = \langle P, W\rangle$ and an interpretation I, the semantics of Π extends from the basic case defined above. Thus, let G_P and G_W be the instantiation of P and W, respectively. Then, let G_W^I be the set

$\{$`(w@l,t1,...,tz) | :~ b1,...,bk,not bk+1,...,not bm. [w@l,t1,...,tz]` $\in G_W$, `b1,...,bk` $\in I$, *and* `bk+1,...,bm` $\notin I\}$.

Moreover, for an integer l, $P_l^I = \sum_{(w@l,t_1,\ldots,t_z)\in G_W^I} w$ if there is at least one tuple in G_W^I whose level is equal to l, and 0 otherwise. Given a program with weak constraints $\Pi = \langle P, W\rangle$, an answer set M for P is said to be dominated by an answer set M' for P, if there exists an integer l such that $P_l^{M'} < P_l^M$ and $P_{l'}^{M'} = P_{l'}^M$ for all integers $l' > l$. An answer set M for P is said to be *optimal* or *optimum* for Π if there is no other answer set M' that dominates M (Calimeri et al., 2020).

3.3.3. Syntactic shortcuts

In the following, `p(1..n).` denotes the set of facts `p(1). ... p(n).` Moreover, we use *choice rules* of the form `{X}`, where `X` is a set of atoms. Choice rules of this kind can be viewed as a syntactic shortcut for the rule `p | p'.` for each `p` $\in$ `X`, where `p'` is a fresh new atom not appearing elsewhere in the program, meaning that the atom p can be chosen as true. Choice rules can also have bounds, e.g., `1 <= {X} <= 1`, and in this case can be seen as a shortcut for the choice rule `{X}` and the rule `:- #count{X} != 1.`

4. Structural and Strong Equivalence of Domain Models

In knowledge engineering for planning and scheduling, there is a growing interest in designing approaches to characterise the similarity of two domain models. In particular, for cases where no assumptions are made on the models being compared. In this line of work, *strong equivalence* of domain models has been informally defined by Shoeeb and McCluskey (2011) as models being identical up to naming. It assumes that there exist bijective mappings between particular elements (e.g., facts, action names). For the STRIPS representation, we formally define *structural and strong equivalence* for two domain models as follows.

Definition 1. *Let $\mathcal{D} = (L, A)$ and $\mathcal{D}' = (L', A')$ be domain models. If there exist bijective mappings $\mathcal{L} : L \to L'$ and $\mathcal{A} : \{name(a) \mid a \in A\} \to \{name(a') \mid a' \in A'\}$ such that for each $a \in A$, if $name(a') = \mathcal{A}(name(a))$, then following conditions hold*

- $pre(a') = \{\mathcal{L}(f) \mid f \in pre(a)\}$
- $del(a') = \{\mathcal{L}(f) \mid f \in del(a)\}$
- $add(a') = \{\mathcal{L}(f) \mid f \in add(a)\}$

then $\mathcal{D}$ *and* $\mathcal{D}'$ *are* **structurally equivalent** *as well as* **strongly equivalent**.

Next, we will construct a *Domain Model Graph* (DMG) which is a directed graph connecting actions with facts in a given domain model. DMG has vertices standing for both facts and action names, and three types of edges referring to preconditions, delete and add effects, respectively. To show that two domain models are strongly equivalent, their respective DMGs have to be isomorphic.

Definition 2. *Let* $\mathcal{D} = (L, A)$ *be a domain model. We say that* $\mathcal{G} = (V, E_{pre}, E_{del}, E_{add})$ *is a* **Domain Model Graph (DMG)** *of* $\mathcal{D}$*, where* $V = L \cup \{name(a) \mid a \in A\}$ *is a set of vertices,* $E_{pre} = \{(name(a), f) \mid f \in pre(a), a \in A\}$*,* $E_{del} = \{(name(a), f) \mid f \in del(a), a \in A\}$ *and* $E_{add} = \{(name(a), f) \mid f \in add(a), a \in A\}$ *are sets of directed edges.*

Theorem 1. *Let* $\mathcal{D} = (L, A)$ *and* $\mathcal{D}' = (L', A')$ *be domain models.* $\mathcal{D}$ *and* $\mathcal{D}'$ *are structurally (and strongly) equivalent if and only if* $\mathcal{G} = (V, E_{pre}, E_{del}, E_{add})$ *being the DMG of* $\mathcal{D}$ *and* $\mathcal{G}' = (V', E'_{pre}, E'_{del}, E'_{add})$ *being the DMG of* $\mathcal{D}'$ *are isomorphic.*

Proof. ($\Rightarrow$): If $\mathcal{D}$ and $\mathcal{D}'$ are structurally (and strongly) equivalent, then there exist bijective mappings $\mathcal{L}$ and $\mathcal{A}$ between facts and action names of both domain models as in Definition 1. We can combine $\mathcal{L}$ and $\mathcal{A}$ into ξ such that for each $f \in L : \xi(f) = \mathcal{L}(f)$ and for each $a \in A : \xi(name(a)) = \mathcal{A}(name(a))$. Hence, ξ is a bijective mapping from V to V'. Then, we can observe that for each $a \in A$ and $f \in pre(a)$ there exists $a' \in A'$ such that $\xi(name(a)) = name(a')$ and $\xi(f) \in pre(a')$. Hence, $(name(a), f) \in E_{pre}$ if and only if $(\xi(name(a)), \xi(f)) \in E'_{pre}$. For E_{del} and E_{add}, it can be proven analogously.

($\Leftarrow$): From Definition 2 and the fact that every action is well defined, we can derive that $X = \{x \mid (x, y) \in E_{del} \cup E_{add}\} = \{name(a) \mid a \in A\}$. If $\mathcal{G}$ and $\mathcal{G}'$ are isomorphic, then there exists a bijective mapping $\xi : V \to V'$ such that $(x, y) \in E_{add}$ if and only if $(\xi(x), \xi(y)) \in E'_{add}$. Hence, we can "split" ξ into two bijective mappings $\mathcal{L}$ and $\mathcal{A}$ such that for each $x \in X : \mathcal{A}(x) = \xi(x)$ and for each $y \in (V \setminus X) : \mathcal{L}(y) = \xi(y)$. We can also observe that for each $a \in A$ and $f \in add(a)$ (it holds that $(name(a), f) \in E_{add}$), there exists $a' \in A'$ such that $name(a') = \mathcal{A}(name(a))$ and $\mathcal{L}(f) \in add(a')$. For $del(a')$ and $pre(a')$, it can be proven analogously. □

The bijective mappings $\mathcal{L}$ and $\mathcal{A}$ can be leveraged for "translating" between problem instances or plans of particular strongly equivalent domain models.

Theorem 1 also shows that determining the structural equivalence of two domain models is at most as hard as determining whether two graphs are isomorphic. The graph isomorphism problem is GI-complete (Köbler et al., 1993) – the problem belongs to NP but it is unknown whether it can be solved polynomially. We would like to note that DMGs have a specific structure, i.e., they are bipartite graphs with edges going only from action to fact vertices. Hence, checking whether the problem of determining structural (and strong) equivalence of domain models is GI-hard (or can be solved polynomially) is still an open question.

As a final note, for STRIPS representation, both structural and strong equivalence are identical as the representation does not contain any numerical attributes.

4.1. Structural and Strong Equivalence of Lifted Domain Models

Since the lifted representation contains free variables, the notions of *structural* and *strong equivalence* have to be extended in order to consider free variables besides the naming. That said, we have to make sure that for each corresponding grounded instance of two strongly (structurally) equivalent lifted domain models, it is the case that these instances are strongly (structurally) equivalent too. Whereas the (bijective) mapping between predicates needs to consider only naming and arity (without loss of generality we assume that free variables in each predicate are distinct), the (bijective) mapping between planning operators also has to distinguish between their parameters (free variables). For the lifted STRIPS representation, we formally define *structural and strong equivalence* for two models as follows. Note that for the lifted STRIPS representation, again, both structural and strong equivalence are identical as the representation does not contain any numeric attributes.

Definition 3. *Let $\mathcal{D} = (P, O)$ and $\mathcal{D}' = (P', O')$ be lifted domain models. If there exist bijective mappings $\mathcal{P} : \{sym(p) \mid p \in P\} \rightarrow \{sym(p') \mid p' \in P'\}$ and $\mathcal{O} : \{name(o) \mid o \in O\} \rightarrow \{name(o') \mid o' \in O'\}$ such that*

- *for each $p \in P$, there exists $p' \in P$ such that $sym(p') \equiv \mathcal{P}(sym(p))$ and $arity(p) = arity(p')$*
- *for each $o \in O$, $arity(name(o)) = arity(\mathcal{O}(name(o)))$ and if $name(o') = \mathcal{O}(name(o))$, then the following conditions hold for some bijective substitution mapping $\chi^o : pars(o) \rightarrow pars(o')$*
  - *$pre(o') = \{(\mathcal{P}(sym(p))(pars(p))|\chi^o) \mid p \in pre(o)\}$*
  - *$del(o') = \{(\mathcal{P}(sym(p))(pars(p))|\chi^o) \mid p \in del(o)\}$*
  - *$add(o') = \{(\mathcal{P}(sym(p))(pars(p))|\chi^o) \mid p \in add(o)\}$*

then $\mathcal{D}$ and $\mathcal{D}'$ are **structurally equivalent** *as well as* **strongly equivalent**.

Next, we will construct a *Lifted Domain Model Graph* (LDMG) which is a labelled directed graph connecting operators with predicates in a given lifted domain model. LDMG has vertices standing for both predicates and operator names, and three types of edges referring to preconditions, delete, and add effects, respectively. Edge labels represent matchings between operators' and predicates' variables. To show that two domain models are strongly equivalent, their respective LDMGs have to be isomorphic. Note that the variable symbols defined in both domain models must be distinct, which can be trivially done by renaming them.

Definition 4. *Let $\mathcal{D} = (P, O)$ be a lifted domain model. We assume, without loss of generality, that all variable symbols defined in the arguments of the predicates from P and the operators from O are distinct. We say that $\mathcal{G} = (V, E_{pre}, E_{del}, E_{add})$ is a* **Lifted Domain Model Graph (LDMG)** *of $\mathcal{D}$, where $V = P \cup \{name(o) \mid o \in O\}$ is a set of vertices, $E_{pre} = \{(name(o), \Theta^o_p, p) \mid (p|\Theta^o_p) \in pre(o), o \in O, p \in P, \Theta^o_p : pars(p) \rightarrow pars(o)\}$, $E_{del} = \{(name(o), \Theta^o_p, p) \mid (p|\Theta^o_p) \in del(o), o \in O, p \in P, \Theta^o_p : pars(p) \rightarrow pars(o)\}$ and $E_{add} = \{(name(o), \Theta^o_p, p) \mid (p|\Theta^o_p) \in add(o), o \in O, p \in P, \Theta^o_p : pars(p) \rightarrow pars(o)\}$ are sets of labelled directed edges.*

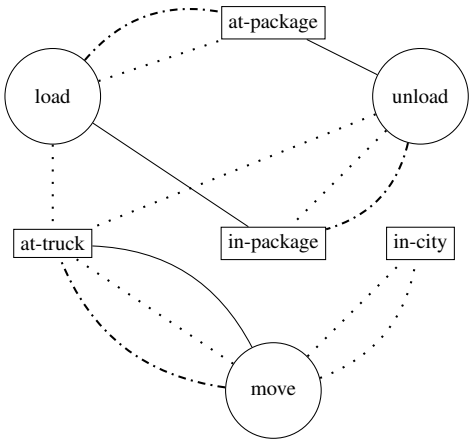

Figure 1: LDMG for the simplified Logistics original domain model. Dotted lines indicate preconditions, dashed lines indicate negative effects, and full lines indicate positive effects. For the sake of readability, we represent operators by circles and predicates by rectangles as well as we omit variables for the same reason.

As an example, Figure 1 shows the LDMG of our Logistics domain in Example 1 that illustrates how operators are connected with predicates (variables are omitted for the sake of readability). To give an example of how variables are bound between operators and predicates, let's assume the load actions with the following parameters: ?t, ?p, ?l (we intentionally consider different parameter names than in Example 1 to highlight the binding of variables). The predicates (at-truck ?Loc ?Truck), (at-package ?Loc ?Pkg) are defined as in Example 1. The substitutions for both predicates (and the load action) are defined as $\Theta^{\text{load}}_{\text{at-truck}} = \{?\text{Loc} \rightarrow ?\text{l}, ?\text{Truck} \rightarrow ?\text{t}\}$ and $\Theta^{\text{load}}_{\text{at-package}} = \{?\text{Loc} \rightarrow ?\text{l}, ?\text{Pkg} \rightarrow ?\text{p}\}$.

Theorem 2. *Let $\mathcal{D} = (P, O)$ and $\mathcal{D}' = (P', O')$ be lifted domain models such that all variable symbols defined in the arguments of the predicates from P and P' and the operators from O and O' are distinct. Let $\mathcal{G} = (V, E_{pre}, E_{del}, E_{add})$ and $\mathcal{G}' = (V', E'_{pre}, E'_{del}, E'_{add})$ be LDMGs of $\mathcal{D}$ and $\mathcal{D}'$, respectively. $\mathcal{D}$ and $\mathcal{D}'$ are structurally and strongly equivalent if and only if $\mathcal{G}$ and $\mathcal{G}'$ are isomorphic with a bijective mapping $\xi : V \rightarrow V'$ such that for each $x \in V : arity(x) = arity(\xi(x))$.*

Proof. ($\Rightarrow$): If $\mathcal{D}$ and $\mathcal{D}'$ are structurally (strongly) equivalent, then there exist bijective mappings $\mathcal{P}$ and $\mathcal{O}$ between predicates and operator names of both domain models as in Definition 3. We can combine $\mathcal{P}$ and $\mathcal{O}$ into ξ analogously to the proof of Theorem 1, except that the "predicate" part of the mapping concerns "whole" predicates instead of only predicate symbols (names) as in Definition 3, i.e., for all $p \in P$, it is the case that $\xi(p) \in P'$ and $\mathcal{P}(sym(p)) = sym(\xi(p))$. Then, we can observe that for each $o \in O$ there exists $o' \in O'$ such that $\xi(\textit{name}(o)) = \textit{name}(o')$, $arity(\xi(\textit{name}(o))) = arity(\textit{name}(o'))$. There also exist substitution mappings χ^o for each $o \in O$ as in Definition 3. Now we can define a bijective substitution mapping $\nu : \bigcup_{o \in O} pars(o) \cup \bigcup_{p \in P} pars(p) \rightarrow \bigcup_{o' \in O'} pars(o') \cup \bigcup_{p' \in P'} pars(p')$ (since variable symbols are distinct) such that for all $o \in O$ and $x \in pars(o)$, $(x|\nu) = (x|\chi^o)$, and for all $p \in P$, $(\mathcal{P}(sym(p))(pars(p))|\nu) = \xi(p)$. Then, if for $o \in O$ and $p \in P$ it is the case that $(p|\Theta^o_p) \in \textit{pre}(o)$, then there exists $o' \in O'$ such that $(\xi(p)|\Theta^{o'}_{p'}) \in \textit{pre}(o')$ while $\Theta^{o'}_{p'} = (\Theta^o_p|\nu)$ (Θ^o_p and $\Theta^{o'}_{p'}$ are defined as in Definition 4). Hence, if $(\textit{name}(o), \Theta^o_p, p) \in E_{pre}$, then $(\xi(\textit{name}(o)), (\Theta^o_p|\nu), \xi(p)) \in E'_{pre}$. For E_{del} and E_{add}, it can be proven analogously.

($\Leftarrow$): If $\mathcal{G}$ and $\mathcal{G}'$ are isomorphic, then there exist a bijective mapping $\xi : V \rightarrow V'$ and a bijective substitution mapping $\nu : \bigcup_{o \in O} pars(o) \cup \bigcup_{p \in P} pars(p) \rightarrow \bigcup_{o' \in O'} pars(o') \cup \bigcup_{p' \in P'} pars(p')$. Bijective mappings $\mathcal{P}$ and $\mathcal{O}$ can be derived from ξ analogously to the proof of Theorem 1 and that $\mathcal{P}$ considers only predicate symbols (names) as in Definition 3. Also, having $\forall x \in V : arity(x) = arity(\xi(x))$ we can derive $arity(\textit{name}(o)) = arity(\mathcal{O}(\textit{name}(o)))$ and that there exists $p' \in P'$ such that $sym(p') \equiv \mathcal{P}(sym(p))$ and $arity(p) = arity(p')$. We can also observe (from the isomorphism of $\mathcal{G}$ and $\mathcal{G}'$) that for each $o \in O$ and $x \in pars(o)$, $(x|\nu) = y$ for some $o' \in O'$ and $y \in pars(o')$. We can define a bijective substitution mapping $\chi^o : pars(o) \rightarrow pars(o')$ such that $(x|\chi^o) = y$, which coincides with ν for o. Hence, we can derive for each $o \in O$ and $p \in \textit{pre}(o)$ that there exists $o' \in O'$ such that $(\mathcal{P}(sym(p))(pars(p))|\chi^o) \in \textit{pre}(o')$. For $\textit{del}(o')$ and $\textit{add}(o')$, it can be proven analogously. □

Lifted domain models are "templates" from which grounded domain models can be generated. Strong equivalence of lifted domain models is more general as it does not depend on particular objects. We show that strongly equivalent lifted domain models can be grounded to strongly equivalent (grounded) domain models if the number of objects used for grounding is the same for both (lifted) models.

Theorem 3. *Let $\mathcal{D} = (P, O)$ and $\mathcal{D}' = (P', O')$ be strongly equivalent lifted domain models, and Obj and Obj' be sets of objects. Let $\mathcal{D}_{Obj}$ ($\mathcal{D}'_{Obj'}$) be grounded domain models obtained by substituting free variables in predicates and operators of $\mathcal{D}$ ($\mathcal{D}'$) by objects Obj (Obj'). If $|Obj| = |Obj'|$, then $\mathcal{D}_{Obj} = (L, A)$ and $\mathcal{D}'_{Obj'} = (L', A')$ are strongly equivalent grounded domain models.*

Proof. Since $\mathcal{D}$ and $\mathcal{D}'$ are strongly equivalent there exists mappings $\mathcal{P} : P \rightarrow P'$ and $\mathcal{O} : \{\textit{name}(o) \mid o \in O\} \rightarrow \{\textit{name}(o') \mid o' \in O'\}$ as in Definition 3. Also, corresponding predicates and operators have the same arity (see Definition 3) and hence $\mathcal{D}$ and $\mathcal{D}'$ have the same number of distinct variable symbols. With $|Obj| = |Obj'|$, we can derive that for each substitution mapping ϕ that maps (all) variable symbols of $\mathcal{D}$ to Obj there exists a corresponding substitution mapping ψ that maps (all) variable symbols of $\mathcal{D}'$ to Obj'. Therefore, we can find bijective mappings $\mathcal{L} : L \rightarrow L'$ and $\mathcal{A} : \{\textit{name}(a) \mid a \in A\} \rightarrow \{\textit{name}(a') \mid a' \in A'\}$ as in Definition 1 such that $\mathcal{L}((p|\phi)) = (p'|\psi)$ and $(\mathcal{O}(name(o))|\phi) = (name(o')|\psi)$ for all $p \in P, o \in O$ and all substitution mappings ϕ and ψ. □

Example 2 (Running example (continuing Example 1)**.** *We can define another domain model that concerns transporting passengers, other than the Logistics domain of Example 1, from one location to another by shuttles. The domain model includes 4 predicates:* (at-shuttle ?Loc ?Shtl), (at-passenger ?Loc ?Psg), (in-passenger ?Psg ?Shtl), (in-city ?Cty ?Loc) *and 3 operators:* embark(?Loc ?Psg ?Shtl), debark(?Loc ?Psg ?Shtl), *and* move(?Cty ?Loc1 ?Loc2 ?Shtl)*. We can observe that the structure of the domain model is identical to the Logistics model apart from the naming of (most of) predicates and operators. Hence, the domain models are strongly equivalent.*

4.2. Beyond Classical STRIPS Planning

The introduced concept of *strong equivalence* can be straightforwardly extended to more expressive domain models in which planning operators are defined by considering more sets of predicates that we defined as *extended planning operators* (see Section 3). To give some

examples, we might consider *negative preconditions*, *action cost*, or *durative actions* in which preconditions and effects are distinguished by a relative time of their occurrence (for details, see the description of durative actions in PDDL2.1 (Fox and Long, 2003)). Those language features are commonly used in realistic applications of planning.

The definition of structural equivalence for extended operators is done in analogy to Definition 3. For strong equivalence, we additionally require that the attributes of "matched" operators are equal.

Definition 5. *Let $\mathcal{D} = (P, O)$ and $\mathcal{D}' = (P', O')$ be lifted domain models, where O and O' are sets of extended planning operators. If there exist bijective mappings $\mathcal{P} : \{sym(p) \mid p \in P\} \rightarrow \{sym(p') \mid p' \in P'\}$ and $\mathcal{O} : \{name(o) \mid o \in O\} \rightarrow \{name(o') \mid o' \in O'\}$ such that*

- *for each $p \in P$, $arity(p) = arity(\mathcal{P}(p))$*
- *for each $o \in O$, $arity(name(o)) = arity(\mathcal{O}(name(o)))$, if $name(o') = \mathcal{O}(name(o))$, then $Z^i(o') = \{(\mathcal{P}(sym(p))(pars(p))|\chi^o) \mid p \in Z^i(o)\} (1 \leq i \leq n)$ holds for some bijective substitution mapping $\chi^o : pars(o) \rightarrow pars(o')$*

then $\mathcal{D}$ and $\mathcal{D}'$ are **structurally equivalent**. *If for each $o \in O$ there exists its structurally equivalent counterpart $o' \in O'$ (as above) it holds that $attr_j(o') = attr_j(o) (1 \leq j \leq m)$, then we say that $\mathcal{D}$ and $\mathcal{D}'$ are* **strongly equivalent**

The definition of extended LDMG (ELDMG) is analogous to Definition 4.

Definition 6. *Let $\mathcal{D} = (P, O)$ be a lifted domain model, where O is a set of extended planning operators. We assume, without loss of generality, that all variable symbols in the arguments of the predicates from P and the operators from O are distinct. We say that $\mathcal{G} = (V, E_1, \ldots, E_n)$ is a* **Extended Lifted Domain Model Graph (ELDMG)** *of $\mathcal{D}$, where $V = P \cup \{name(o) \mid o \in O\}$ is a set of vertices, $E_i = \{(name(o), \Theta^o_p, p) \mid (p|\Theta^o_p) \in Z^i(o), o \in O, p \in P, \Theta^o_p : pars(p) \rightarrow pars(o)\}$ with $1 \leq i \leq n$ are sets of labelled directed edges.*

Determining the structural equivalence of domain models with extended planning operators can be reduced to the problem of determining whether their ELDMGs are isomorphic. Thus, we can extend Theorem 2 by considering extended planning operators. For strong equivalence, we have to ensure that all attributes of "matched" operators are equal.

Theorem 4. *Let $\mathcal{D} = (P, O)$ and $\mathcal{D}' = (P', O')$ be lifted domain models, where O and O' are sets of extended planning operators such that all variable symbols defined in the arguments of the predicates from P and P' and the operators from O and O' are distinct. Let $\mathcal{G} = (V, E_1, \ldots, E_n)$ and $\mathcal{G}' = (V', E'_1, \ldots, E'_n)$ be ELDMGs of $\mathcal{D}$ and $\mathcal{D}'$, respectively. $\mathcal{D}$ and $\mathcal{D}'$ are structurally equivalent if and only if $\mathcal{G}$ and $\mathcal{G}'$ are isomorphic with a bijective mapping $\xi : V \rightarrow V'$ such that for each $x \in V : arity(x) = arity(\xi(x))$. $\mathcal{D}$ and $\mathcal{D}'$ are strongly equivalent if and only if they are structurally equivalent and for each $o \in O$ and $o' \in O'$ such that $name(o') = \xi(name(o))$ it is the case that $attr_j(o') = attr_j(o), 1 \leq j \leq m$.*

Proof. The claim directly follows from the proof of Theorem 2, since it can be generalised for more sets of predicates and sets of edges, and from Definition 5 (equality of corresponding attributes). □

We would like to note that attributes such as action cost or duration can be expressed (and often are) by lifted function symbols specified by their unique name and a set of variable symbols (parameters), analogously to predicates in the lifted STRIPS representation. Hence, we can represent these function symbols as vertices in ELDMGs and introduce corresponding types of edges (e.g. "cost" edges or "duration" edges) to connect operators with these vertices and directly leverage Theorem 4 to determine strong equivalence. Note that such an approach may not be applicable if action cost or duration (or any other attribute) is specified by arithmetic expressions involving combinations of multiple function symbols and/or constants.

In cases where a domain model contains operators having both numeric and functional cost (or duration), we can consider both attributes and "cost" (or "duration") edges. For an operator with a numeric cost (or duration), we set the attribute to the corresponding value. For an operator with a functional cost (or duration), we include the corresponding "cost" (or "duration") edge while setting the cost (or duration) attribute to zero.

In cases where a domain model contains both durative actions and STRIPS operators, we need to consider both types of operators as extended operators with all predicate sets and attributes as for durative actions. We can map preconditions, delete, and add effects of a STRIPS operator to "at start" preconditions, "at start" delete, and "at end" add effects, respectively, of an extended operator, while having a duration attribute set to zero.

Example 3. *Let us recall both logistics domain models from Example 2. Now, we assume that the operators* load(?Loc ?Pkg ?Truck)*,* unload(?Loc ?Pkg ?Truck)*, and* move(?Cty ?Loc1 ?Loc2 ?Truck) *are temporal (according to PDDL 2.1 semantics) such that durations of loading and unloading are constant while duration of the* move *operator is represented by a numeric fluent* (distance ?Loc1 ?Loc2)*. Analogously, we can define temporal operators* embark(?Loc ?Psg ?Shtl)*,* debark(?Loc ?Psg ?Shtl)*, and* move(?Cty ?Loc1 ?Loc2 ?Shtl) *for the other model with durations of* embark *and* debark *being constant and duration of the* move *operator represented by a numeric fluent* (distance ?Loc1 ?Loc2)*. Both models (with temporal operators) can be strongly equivalent as they differ only in naming if and only if the durations for loading/unloading are the same as for embarking/debarking. If, for instance, embarking/debarking is faster than loading/unloading packages, then the models are not strongly equivalent but are structurally equivalent.*

5. Domain Model Similarity

The notions of *structural and strong equivalence* that were introduced in the previous section are important building blocks for quantifying how similar two domain models are. In particular, *domain model similarity* enumerates how many manipulations (adding/modifying an element in either of the models) are needed to make the models strongly equivalent. This notion carries both syntactical and semantical meanings: on the syntactical side, it quantifies the modifications needed to perfectly align two models. On the semantical side, an analysis of the changes needed can shed some light into the ability of the models to generate similar sets of solutions.

In principle, we build upon the theoretical results in the previous section that reduce the problem of determining structural equivalence of two domain models into the problem of determining whether (E)LDMGs of these domain models are isomorphic. Then, *domain model similarity* can be measured by a variant of *edit distance* of (E)LDMGs as we present in this section.

Initially, we define the notion of *submodel* that describes the relation between (extended) domain models based on the subgraph relation between their (E)LDMGs.

Definition 7. *Let $\mathcal{D}$ and $\mathcal{D}'$ be (extended) domain models. We say that $\mathcal{D}'$ is a* **submodel** *of $\mathcal{D}$ if the (E)LDMG of $\mathcal{D}'$ is a subgraph of the (E)LDMG of $\mathcal{D}$. We say that an (extended) domain model $\mathcal{D}''$ is a* **strongly (structurally) equivalent submodel** *of $\mathcal{D}$ if $\mathcal{D}''$ is strongly (structurally) equivalent with $\mathcal{D}'$ (being a submodel of $\mathcal{D}$).*

Often, (extended) domain models share the same structure only partially, hence, they share common structurally (strongly) equivalent submodels.

Definition 8. *Let $\mathcal{D}_1$ and $\mathcal{D}_2$ be (extended) domain models. We say that submodels $\mathcal{D}'_1$ and $\mathcal{D}'_2$ of $\mathcal{D}_1$ and $\mathcal{D}_2$, respectively, are* **common strongly (structurally) equivalent submodels** *of $\mathcal{D}_1$ and $\mathcal{D}_2$ if and only if $\mathcal{D}'_1$ and $\mathcal{D}'_2$ are strongly equivalent.*

The following proposition makes the connection between common structurally equivalent submodels and common isomorphic subgraphs of models' respective (E)LDMGs.

Proposition 1. *Let $\mathcal{D}'_1$ and $\mathcal{D}'_2$ be submodels of (extended) domain models $\mathcal{D}_1$ and $\mathcal{D}_2$, respectively. It holds that $\mathcal{D}'_1$ and $\mathcal{D}'_2$ are common structurally equivalent submodels of $\mathcal{D}_1$ and $\mathcal{D}_2$ if and only if the (E)LDMG of $\mathcal{D}'_1$ and the (E)LDMG of $\mathcal{D}'_2$ are common isomorphic subgraphs of the (E)LDMGs of $\mathcal{D}_1$ and $\mathcal{D}_2$, respectively.*

Proof. The claim of the proposition is directly implied from the definition of common isomorphic subgraphs (see the Background Section) and Theorems 2 and 4. □

We can generalise the notion of *common structurally equivalent submodels* for multiple domain models. The key idea is to identify a *common core*, i.e., submodels of the models such that every two submodels are structurally equivalent. Note that we can straightforwardly extend Proposition 1 such that it applies to multiple submodels.

Definition 9. *Let $\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_n$ be (extended) domain models. We say that submodels $\mathcal{D}'_1, \mathcal{D}'_2, \ldots \mathcal{D}'_n$ of $\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_n$, respectively, are a* **common core** *for $\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_n$ if and only if $\mathcal{D}'_i$ and $\mathcal{D}'_j$, for all $1 \leq i, j \leq n$, are structurally equivalent.*

The next definition connects our variant of the edit distance of graphs and the distance of domain models. In the case of lifted domain models (or extended domain models without attributes), the notion of *graph edit distance* corresponds with the notion of *distance of domain models*.

Definition 10. *Let $\mathcal{D}_1$ and $\mathcal{D}_2$ be domain models and $\mathcal{G}_1$ and $\mathcal{G}_2$ be their LDMGs, respectively. We define a dist function representing the* **distance** *between $\mathcal{D}_1$ and $\mathcal{D}_2$ as dist$(\mathcal{D}_1, \mathcal{D}_2) =$ dist$(\mathcal{G}_1, \mathcal{G}_2)$.*

The notion of distance between two lifted domain models is determined through the concept of *elementary operations* that will be defined later for extended domains, and will correspond to the (minimum) number of modifications needed by these two models to make them strongly equivalent. That corresponds to adding vertices and edges to the LDMGs of these two models.

We would like to emphasise that we do not explicitly distinguish operator and predicate nodes. We can observe that an operator has to have at least one add or delete effect and hence the corresponding vertex in the underlying LDMG has to have at least one outgoing "del" or "add" edge. Note, again, that we assume that all operators are well defined. On the other hand, each predicate node has no outgoing edge.

Example 4 (Example 1 cont'd)**.** *Let us simplify the model introduced in Example 2 by removing the (in-city ?Cty ?Loc) predicate. The simplified model is a submodel of the original one. Now, let us add a macro-operator move-load(?Pkg ?Truck ?Loc1 ?Loc2 ?Cty) encapsulating the sequence of move and load operators into the simplified model. The simplified model is a submodel of the "macro" model. Finally, we can compare the "macro" Logistics model with the "passenger" model from Example 2. The models are not strongly equivalent. We can, on the other hand, find their maximum common strongly equivalent submodels, i.e., the simplified Logistic model and the "passenger" model which is simplified by removing (in-city ?Cty ?Loc).*

For more expressive domain models (as described in Section 4.2), the above concepts of domain model similarity can be straightforwardly extended for models in which operators do not contain attributes. The existence of attributes, however, affects the notion of distance (of domain models) as it does not only depend on our variant of edit distance between ELDMGs, but also on the equality of attributes of "matched" operators.

In contrast to Definition 10, we have to take into account differences in attributes and, hence, the graph edit distance of ELDMGs is not necessarily equal to the distance of the extended domain models (with attributes). The distance between extended domain models, formally defined as follows, has to take into account both common structurally equivalent submodels and (in)equal attributes of corresponding operators. Note that for the sake of clarity, we will abuse the notation by considering the (bijective) mapping between operators rather than operator names (and parameters) as in Definition 5.

Definition 11. *Let $\mathcal{D}_1$ and $\mathcal{D}_2$ be extended domain models and $\mathcal{G}_1$ and $\mathcal{G}_2$ be their ELDMGs, respectively. Let $\mathcal{D}'_1$ and $\mathcal{D}'_2$ be common structurally equivalent submodels of $\mathcal{D}_1$ and $\mathcal{D}_2$, and $\mathcal{G}'_1$ and $\mathcal{G}'_2$ be their ELDMGs, respectively, with ξ as the bijective mapping from vertices of $\mathcal{G}'_1$ to vertices of $\mathcal{G}'_2$ making $\mathcal{G}'_1$ and $\mathcal{G}'_2$ isomorphic. We define a function diff such as $diff(D_1, D_2, D'_1, D'_2) = dist(\mathcal{G}'_1, \mathcal{G}'_2) + |\{(o, attr_i) \mid o \text{ defined in } D_1, attr_i(o) \neq attr(\xi(o))\}|$. Then, we define a dist function representing the* **distance** *between $\mathcal{D}_1$ and $\mathcal{D}_2$ as $dist(\mathcal{D}_1, \mathcal{D}_2)$ being the minimum of $diff(D_1, D_2, D'_1, D'_2)$ with D'_1, D'_2 being common structurally equivalent submodels of D_1, D_2.*

The notion of distance between two (extended) lifted domain models determines a minimum number of *elementary operations* to modify these two models to make them strongly equivalent. That corresponds to adding vertices and edges to the (E)LDMGs of these two models. A single elementary operation performed on either model reduces the distance between the models by one.

Definition 12. *Let $\mathcal{D} = (P, O)$ be an (extended) lifted domain model and $\mathcal{G} = (V, E_1, E_2, \ldots, E_n)$ its ELDMG, and let $\mathcal{D}' = (P', O')$ be another (extended) lifted domain model and $\mathcal{G}' = (V', E'_1, E'_2, \ldots, E'_n)$ its ELDMG. We define the following* ***elementary operations*** *over $\mathcal{D}$ and $\mathcal{G}$ (resp. $\mathcal{D}'$ and $\mathcal{G}'$) such that for $\mathcal{D}''$, the resulting model after performing a single elementary operation, it is the case that $dist(\mathcal{D}'', \mathcal{D}') = dist(\mathcal{D}, \mathcal{D}') - 1$ (resp. $dist(\mathcal{D}, \mathcal{D}'') = dist(\mathcal{D}, \mathcal{D}') - 1$):*

(1) *Add o into O iff o is added into V and (o, p) is added into E_i for some p and i (resp. add o' into O' iff o' is added into V' and (o', p') is added into E'_i for some p' and i).*

(2) *Add p into P iff p is added into V and no edge from p is added to E_i for any i (resp. add p' into P' iff p' is added into V' and no edge from p' is added to E'_i for any i).*

Algorithm 1: Comparing Domain Models

Input : A graph $\mathcal{G}_1$ and graph $\mathcal{G}_2$ and a boolean $Preprocessing$
Output: Differences between $\mathcal{G}_1$ and $\mathcal{G}_2$

```
if Preprocessing is true then
    Π := preprocessing(G1, G2);
else
    Π := nopreprocessing(G1, G2);
Π_f := Π ∪ Π' ∪ Π'' ;                    // Π'' reported in Listing 2
A := ASPSolver(Π_f);
ProcessSolution(A) ;
```

(3) *Add p into $Z^i(o)$ iff (o,p) is added into E_i (resp. add p' into $Z'^i(o')$ iff (o',p') is added into E'_i).*
(4) *Remove p from $Z^i(o)$ iff (o,p) is removed from E_i (resp. remove p' from $Z'^i(o')$ iff (o',p') is removed from E'_i).*
(5) *Set $attr(o)$ ($o \in O$) to $attr(\xi(o))$ (resp. $attr(o')$ ($o' \in O'$) to $attr(\xi(o'))$), where ξ is a bijective mapping from the vertices of $\mathcal{G}$ to the vertices of $\mathcal{G}'$ (resp. from the vertices of $\mathcal{G}'$ to the vertices of $\mathcal{G}$).*

We would like to emphasise that we do not explicitly distinguish operator and predicate nodes. We can observe that an operator has to have at least one add or delete effect and hence the corresponding vertex in the underlying ELDMG has to have at least one outgoing edge. Note, again, that we assume that all operators are well defined. On the other hand, each predicate node has no outgoing edge.

The problem of *domain model similarity* is to find the minimum number of elementary operations such that the models become strongly equivalent.

Definition 13. *Let $\mathcal{D}_1$ and $\mathcal{D}_2$ be (extended) lifted domain models. The **domain model similarity problem** is to find a minimum number of elementary operations (over $\mathcal{D}_1$ and $\mathcal{D}_2$) such that the resulting models after performing the elementary operations are strongly equivalent.*

It is known that the problem of graph edit distance is NP-hard (Zeng et al., 2009). Due to the specific structure of (E)LDMGs, again, the question of whether determining the distance between domain models is NP-hard or easier (in P) is still open.

6. Comparing Domain Models via ASP

In this section, we describe our approach based on Answer Set Programming (ASP): following the previous organization, we first present the approach on classical STRIPS domains, then on extended domains including, e.g., durative actions, in two separate subsections.

6.1. ASP-based comparison on classical domains

Following the theory presented in previous sections, we implemented the ASP-based approach depicted in Algorithm 1. The algorithm can also work with extended domains including durative actions and action costs which will be presented in the next subsection. The algorithm receives two LDMGs (referred to as $\mathcal{G}_1$ and $\mathcal{G}_2$) and a Boolean variable (referred to

Algorithm 2: Process Solution

Input : An optimal ASP solution A
Output: The differences between $\mathcal{G}_1$ and $\mathcal{G}_2$

```
1  foreach ver(id, add, g, name, ...) ∈ A do
2      Add vertex [[name]] in [[g]];
3  foreach edge(add, g, n1, n2, t, l) ∈ A do
4      Add [[t]] edge with label [[l]] from vertex [[n1]] to vertex [[n2]] in [[g]];
5  foreach edge(del, g, n1, n2, t, l) ∈ A do
6      Del [[t]] edge with label [[l]] from vertex [[n1]] to vertex [[n2]] in [[g]];
7  foreach val(attr, id, add, g, t, v, p) ∈ A do
       // attr can be duration or cost
8      if t ∈ ℕ then
9          Add [[attr]] with id [[id]] of type [[t]] with value [[v]] in [[g]];
10     else
11         Add [[attr]] with id [[id]] of type [[t]] linked to function [[v]] in [[g]];
12 foreach map(n1, n2, e1, e2) ∈ A do
13     Map vertex [[n1]] of [[G2]] to vertex [[n2]] of [[G1]];
14     if e1 ≠ e2 then
15         Remapping [[e1]] to [[e2]];
16 foreach mapValue(n1, n2, e1, e2) ∈ A do
17     Map value [[n1]] of [[G1]] to value [[n2]] of [[G2]];
18     if e1 ≠ e2 then
19         Remapping [[e1]] to [[e2]];
```

as $Preprocessing$) as input, and prints $dist(\mathcal{G}_1, \mathcal{G}_2)$, i.e., the minimum number of elementary operations to the graphs to make them isomorphic as output. In the following, we assume that the number of vertices representing an operator (resp. a predicate) of $\mathcal{G}_1$ is less than or equal to the number of vertices representing an operator (resp. a predicate) of $\mathcal{G}_2$, and we perform the elementary operations only on $\mathcal{G}_1$. The idea of the algorithm is to (i) first create an ASP program Π starting from the input graphs, by either using the function at line 2 or 4 depending on the value of $Preprocessing$, then (ii) Π is extended with an ASP program used to compute the distance between $\mathcal{G}_1$ and $\mathcal{G}_2$, and an ASP solver is used to compute an (optimal) answer set of Π. Finally, (iii) such an answer set is processed to produce human-readable instructions of the elementary operations needed to make the two graphs isomorphic. Steps (i)-(iii) are described in the following, first for the case in which the value of the variable $Preprocessing$ is false, then for true. Finally, note that ASP solvers already permit enumerating all (optimal) answer sets. Therefore, Algorithm 1 can be easily extended to deal with the generation of multiple solutions by applying line 7 to each computed answer set.

Preprocessing disabled. The ASP program for step (i) operates on atoms over the predicates ver, $edge$, and map, as follows:

- $ver(id, status, graph, name, type, parameters, changes)$ denotes the vertices of the input graphs, where id is a unique identifier of the vertex, $status$ indicates if the vertex

was in the input graph ($orig$) or if it must be added (add), $graph$ indicates the graph of the vertex (between $\mathcal{G}_1$ and $\mathcal{G}_2$), $name$ is the name of the vertex, $type$ indicates if the vertex is a predicate or an operator, $parameters$ is a string representing the parameters of the predicate, and $changes$ indicates if (and how) the parameters of the vertex must be changed for a correct match.

- $edge(status, graph, id\_ver1, id\_ver2, type, label)$ denotes the edges of the input graphs, where $status$ and $graph$ are as the ones of ver, id_ver1 and id_ver2 are the identifiers of the vertices connected by the edges, $type$ indicates if the edge is in E_{pre}, E_{del}, or E_{add}, respectively, and $label$ is the label of the edge.

- $map(id\_ver1,id\_ver2,params1,params2)$ denotes the mapping from vertices of the different graphs, where id_ver1 and id_ver2 belong to $\mathcal{G}_2$ and $\mathcal{G}_1$, respectively, whereas $params1$ and $params2$ denote the parameters of the two matched vertices, respectively.

Since the value of $Preprocessing$ is false, the function $nopreprocessing(\mathcal{G}_1, \mathcal{G}_2)$ at line 4 of Algorithm 1 for classical STRIPS domains is invoked, and creates the following rules:

1. $ver(id,orig,gr_i,name,t,"x_1,\ldots,x_k","x_1,\ldots,x_k")$.
   for each vertex $name(x_1,\ldots,x_k)$ in $\mathcal{G}_i(i \in \{1,2\})$ representing a vertex of type t, where t is $predicate$ or $operator$, and id is an identifier of the vertex;
2. $edge(orig,gr_i,v_1,v_2,x,label)$.
   for each edge (v_1, v_2) in E_x(with $x \in \{pre, del, add\}$) of the graph $\mathcal{G}_i$(with $i \in \{1,2\}$), where $label$ is the label of the edge;
3. $\{ver(id,add,gr_1,name,t,"x_1,\ldots,x_k","x_1,\ldots,x_k")\}$.
   for each vertex $name(x_1,\ldots,x_k)$ in $\mathcal{G}_2$ of type t, such that there is no corresponding vertex $name(x_1,\ldots,x_k)$ of type t in $\mathcal{G}_1$; i.e., G_1 contains no vertex with the same name, type, and ordered parameter representation. This condition only avoids generating a duplicate vertex-insertion candidate;
4. $\{elementToElement("x_1,\ldots,x_k","y_1,\ldots,y_z",t)\}$.
   for each mapping of terms/operators of type t with the set of the parameters $x_1,\ldots,x_k$ different from the set of the parameters $y_1,\ldots,y_z$.

In particular, rules of type 4 are needed for the solver to create new added vertices with the correct types and terms, and to generate explicitly all the possible mappings for all terms/-operator combinations; then, it is left to the solver to guess the combination.

The program Π produced before is combined with another ASP encoding Π' in line 5 (step (ii)), which is described next (note that Π'' is empty since we are dealing with classical STRIPS domains). In particular, Π' is made of the following rules (r_1–r_{11}):

```
r1  {map(ID1,ID2,N1,N2):ver(ID2,_,gr1,_,T,P,N1)} = 1 :- ver(ID1,_,gr2,_,T,P,N2).
r2  :- ver(ID,_,gr1,_,_,_,_), #count{X: map(X,ID,_,_)} != 1.
r3  :- #sum{1,ID: ver(ID,_,gr2,_,_,_,_); -1,ID: ver(ID,_,gr1,_,_,_,_)} != 0.
r4  edge(add,gr1,X2,X4,L,R) :- map(X1,X2,_,_), map(X3,X4,_,_), edge(orig,gr2,X1,X3,L,R), not
      edge(orig,gr1,X2,X4,L,R).
r5  edge(del,gr1,X2,X4,L,R) :- map(X1,X2,_,_), map(X3,X4,_,_), edge(orig,gr1,X2,X4,L,R), not
      edge(orig,gr2,X1,X3,L,R).
r6  ver(ID1,orig,gr1,NAME,T,P1,(P1, P2)) :- ver(ID1,orig,gr1,NAME,T,P2,_),
      elementToElement(P2,P1,T).
r7  :~ ver(ID,orig,G,_,_,_,N1), ver(ID,orig,G,_,_,_,N2), N1 != N2. [1@1,ID]
r8  :~ elementToElement(P2,P1,T). [1@1,P2,P1,T]
r9  :~ ver(ID,add,G,_,_,_,_). [1@2,ID,G]
r10 :~ edge(add,G,N1,N2,L,R). [1@2,G,N1,N2,L,R]
```

```
r11 :~ edge(del,G,N1,N2,L,R). [1@2,G,N1,N2,L,R]
```

Listing 1: ASP Program Π'.

Rules r_1 and r_2 associate each vertex of $\mathcal{G}_2$ to exactly one vertex of the same type of $\mathcal{G}_1$. Rule r_3 ensures that the number of vertices of the two graphs is the same. Rules r_4 and r_5 generate the edge modifications required by the mapping: r_4 introduces edges to be added to $\mathcal{G}_1$, while r_5 identifies edges to be removed from $\mathcal{G}_1$. Rule r_6 is needed to change the parameters to the original vertices. Finally, weak constraints r_7, r_8, r_9, r_{10}, and r_{11} minimise the number of vertices with different parameters, the number of mappings between elements, the number of added vertices, the number of added edges, and the number of removed edges, respectively. Observe that weak constraints r_9, r_{10}, r_{11} have a higher priority level than r_7 and r_8, that is, preserving the same number of vertices and edges has a higher priority.

An ASP solver is then executed at line 6 of Algorithm 1 on the resulting program, and produces the (optimal) solution A.

The final step of our algorithm is presented in Algorithm 2, where $ProcessSolution(A)$ is presented. It takes as input an (optimal) answer set, which is processed to produce human-readable instructions of the elementary operations needed to make the two graphs isomorphic (lines 1–2 and 12–15 with the mapping only on vertexes. Given we are considering classical STRIPS domains, including negative preconditions, lines 7–11 and 16–19 will never be executed.)

Note that in Algorithm 2, text in *italics* denotes the literal text generated in the human-readable report. Expressions enclosed in [[...]] denote placeholders that are instantiated with the corresponding values extracted from the answer set A.

Preprocessing enabled. If, instead, the value of the input variable $Preprocessing$ is true, the function $preprocessing(\mathcal{G}_1, \mathcal{G}_2)$ at line 2 of Algorithm 1 for classical STRIPS domains is invoked. In step (i) it creates a program Π made of rules of type 1-3, while rules of type 4 are substituted by:

4'. $\{ver(id,orig,gr_1,name,t,"y_1,\ldots,y_z",("y_1,\ldots,y_z","x_1,\ldots,x_k"))\}$.
for each vertex $name(x_1,\ldots,x_k)$ in $\mathcal{G}_1$ of type t and each vertex $name(y_1,\ldots,y_z)$ in $\mathcal{G}_2$ of type t having a different parameter representation. The sixth argument is the target parameter representation, so that the candidate can be selected by r_1, while the last argument records the target and original representations, as in r_6.

In practice, rules of type 4' allow generating only the meaningful combinations of vertices and terms in a graph via imperative programming by iterating on the meaningful mapping of terms/operators. These combinations are added as choice rules and can be selected by the solver as alternative parameter representations of original vertices.

Further, Π' is made of the rules reported in Listing 1, without considering the ones containing the atom $elementToElement(\cdot)$, i.e., rules r_6 and r_8, since they are no longer needed. Finally, the last step proceeds as in the case where $Preprocessing$ is false.

We now establish the correspondence between the answer sets of $\Pi \cup \Pi'$ and the transformations represented by the encoding. Since the weak constraints use two priority levels, we state the optimisation criterion explicitly.

Proposition 2. *For either value of $Preprocessing$, the answer sets of $\Pi \cup \Pi'$ encode exactly the transformations of $\mathcal{G}_1$ that make it isomorphic to $\mathcal{G}_2$. Moreover, its optimal answer sets*

encode exactly the transformations that lexicographically minimise the number of structural changes and then the cost of parameter remappings.

Proof. We first show that $\Pi \cup \Pi'$ makes available every vertex candidate that may be selected by r_1. Rules of type 3 generate a vertex-insertion candidate for every vertex of $\mathcal{G}_2$ that is not already represented in $\mathcal{G}_1$. Consider now an original vertex of $\mathcal{G}_1$ and a vertex of $\mathcal{G}_2$ of the same type. If their parameter representations coincide, the original fact generated by rule type 1 is already available to r_1. Otherwise, we distinguish two cases:

- when preprocessing is disabled, a rule of type 4 makes the corresponding atom of the form $elementToElement$ available and r_6 derives the required ver atom with the target parameter representation;
- when preprocessing is enabled, the same ver candidate is generated directly by a rule of type 4'.

Thus, both variants of Π provide every vertex insertion and parameter remapping that can be required by a type- and parameter-compatible mapping from $\mathcal{G}_2$ to $\mathcal{G}_1$.

For soundness, let A be an answer set of $\Pi \cup \Pi'$, and let V_i^A be the set of vertex identifiers occurring in atoms of the form $ver(id, \_, gr_i, \ldots)$ in A. The facts generated by rules of types 1 and 2 encode all original vertices and edges. A selected rule of type 3 introduces a new vertex identifier, whereas alternative parameter representations generated by type 4 and r_6, or by type 4', retain the identifier of the original vertex and therefore do not introduce distinct graph vertices. The representation used in the transformation is the one occurring in the selected mapping atom.

Constraint r_3 enforces $|V_1^A| = |V_2^A|$. For every identifier $id_1 \in V_2^A$, rule r_1 selects exactly one atom

$$map(id_1, id_2, n_1, n_2)$$

whose endpoints have the same type and parameter representation. Conversely, r_2 requires every $id_2 \in V_1^A$ to occur in exactly one selected mapping atom. Hence, the mapping atoms in A define a bijection $\mu_A : V_2^A \to V_1^A$ that preserves vertex types and the selected parameter representations.

It remains to consider the edges. Rule r_4 adds every edge of $\mathcal{G}_2$ whose corresponding edge under μ_A is missing from $\mathcal{G}_1$. Conversely, rule r_5 removes every edge of $\mathcal{G}_1$ whose corresponding edge under μ_A^{-1} is missing from $\mathcal{G}_2$. Both rules preserve the edge type and label. Therefore, after applying the additions and deletions encoded by A, an edge (u, v) of type t and label ℓ belongs to $\mathcal{G}_2$ if and only if the corresponding edge $(\mu_A(u), \mu_A(v))$, with the same type and label, belongs to the transformed graph $\mathcal{G}_1^A$. Hence, μ_A is an isomorphism from $\mathcal{G}_2$ to $\mathcal{G}_1^A$. This proves soundness.

For completeness, let τ transform $\mathcal{G}_1$ into a graph $\mathcal{G}_1^\tau$ isomorphic to $\mathcal{G}_2$, and let $\mu : V_2 \to V_1^\tau$ witness the isomorphism. By the candidate-generation argument above, every vertex insertion and parameter remapping used by τ is available in $\Pi \cup \Pi'$. Select the corresponding type 3 candidates. If preprocessing is disabled, select the required type 4 atoms, from which r_6 derives the remapped vertex representations; if it is enabled, select the corresponding type 4' candidates directly. Then select in r_1 the mapping atoms induced by μ. Since μ is bijective, constraints r_2 and r_3 are satisfied. Rules r_4 and r_5 derive exactly the edge additions and deletions in the symmetric difference of the edge relations under μ. Once the choices are fixed, the remaining normal rules form a stratified program whose negative literals refer only

to the input-edge facts. Its least model therefore extends the selected atoms to an answer set encoding τ. This proves completeness.

Finally, let $c_s(A)$ be the total weight contributed at priority level 2 by r_9–r_{11}. Since all their weights are one and their tuples identify individual operations, $c_s(A)$ is the number of selected vertex insertions and edge additions or deletions. Let $c_p(A)$ be the total weight contributed at priority level 1 by r_7 and, when preprocessing is disabled, r_8. Weak constraints do not change the set of answer sets. Since priority level 2 is optimised before priority level 1, they select precisely the answer sets with lexicographically minimum cost

$$cost(A) = \langle c_s(A), c_p(A) \rangle.$$

Together with soundness and completeness of the feasible answer sets, this proves the optimality claim. □

The primary optimisation criterion, represented by priority level 2, coincides with Definition 13: rules r_9–r_{11} minimise the number of vertex insertions, edge additions, and edge deletions. The weak constraints at priority level 1 do not contribute to this distance; they are used only as a secondary criterion to prefer, among transformations having the same minimum number of elementary operations, those requiring fewer parameter remappings.

The behaviour of Algorithm 2 is exemplified in the following Example 5 together with a full execution of Algorithm 1.

Example 5 (Continuing Example 2)**.** *Let $\mathcal{G}_1$ and $\mathcal{G}_2$ be the LDMGs of the Logistics domains considered in Example 2 and the "macro" variant with the $incity$ predicate (see Example 4), respectively. For the sake of compactness, we shorten the names of variables of STRIPS predicates and operators to a single capital letter. Thus, $\mathcal{G}_2$ consists of the vertices of $\mathcal{G}_1$ extended with a vertex of the form $moveload(?P, ?T, ?L1, ?L2, ?C)$.*

Moreover, $E_t^2 = E_t^1 \cup E'_t$ ($t \in \{pre, del, add\}$), where

- $E'_{pre} := \{(moveload,\ ?T =?T,\ ?L1 =?L,\ attruck),\ (moveload,\ ?L1 =?L,\ ?C =?C,\ incity),\ (moveload,\ ?L2 =?L,\ ?C =?C,\ incity),\ (moveload,\ ?P =?P,\ ?L1 =?L,\ atpackage)\}$*; and*
- $E'_{add} := \{(moveload,\ ?T =?T,?L1 =?L,\ atpackage),\ (moveload,\ ?P =?P,\ ?T =?T,\ inpackage)\}$*; and*
- $E'_{del} := \{(moveload,\ ?T =?T,\ ?L1 =?L,\ attruck),\ (moveload,\ ?P =?P,\ ?L1 =?L,\ atpackage)\}$.

Note that rules 1 and 2 encode the vertices and the edge, for instance $ver(0,\ orig,\ gr_1,\ load,\ operator,\ "L, P, T",\ "L, P, T")$ represents the vertex $load$ of $\mathcal{G}_1$, where 0 is a unique identifier of the vertex, and $"L, P, T"$ corresponds to the (ordered) parameters. Moreover, there is only one rule of type 3, i.e., $\{ver(7,\ add,\ gr_1,\ moveload,\ operator,\ "C, L, L, P, T",\ "C, L, L, P, T")\}$, since there is only one vertex in $\mathcal{G}_2$ that is not in $\mathcal{G}_1$. Finally, in case $Preprocessing$ is false an excerpt of the generated rules of type 4 is:

$\{elementToElement(\text{"}C, L, L, T\text{"}, \text{"}L, P, T\text{"}, operator)\}.$
$\{elementToElement(\text{"}C, L, L, T\text{"}, \text{"}C, L, L, P, T\text{"}, operator)\}.$
$\{elementToElement(\text{"}L, P, T\text{"}, \text{"}C, L, L, P, T\text{"}, operator)\}.$
$\{elementToElement(\text{"}P, T\text{"}, \text{"}L, T\text{"}, predicate)\}.$
$\{elementToElement(\text{"}P, T\text{"}, \text{"}L, P\text{"}, predicate)\}.$
$\{elementToElement(\text{"}P, T\text{"}, \text{"}C, L\text{"}, predicate)\}.$
$\{elementToElement(\text{"}L, T\text{"}, \text{"}L, P\text{"}, predicate)\}.$
$\{elementToElement(\text{"}L, T\text{"}, \text{"}C, L\text{"}, predicate)\}.$
$\{elementToElement(\text{"}L, P\text{"}, \text{"}C, L\text{"}, predicate)\}.$

while, in case it is true, an excerpt of the rules of type 4' created is in the following:

$\{ver(0, orig, gr_1, load, operator, \text{"}C, L, L, T\text{"}, (\text{"}C, L, L, T\text{"}, \text{"}L, P, T\text{"}))\}.$
$\{ver(0, orig, gr_1, load, operator, \text{"}C, L, L, P, T\text{"}, (\text{"}C, L, L, P, T\text{"}, \text{"}L, P, T\text{"}))\}.$
$\ldots$
$\{ver(6, orig, gr_1, incity, predicate, \text{"}L, P\text{"}, (\text{"}L, P\text{"}, \text{"}C, L\text{"}))\}.$
$\{ver(6, orig, gr_1, incity, predicate, \text{"}L, T\text{"}, (\text{"}L, T\text{"}, \text{"}C, L\text{"}))\}.$
$\{ver(6, orig, gr_1, incity, predicate, \text{"}P, T\text{"}, (\text{"}P, T\text{"}, \text{"}C, L\text{"}))\}.$

Then, the ASP solver is executed (line 6 of Algorithm 1) on the resulting program extended with Π' (line 5). Further, Algorithm 2 analyses the output (from line 2 on) and produces the instructions about the modifications to be done, an excerpt (i.e., one for each type of instruction) is in the following:

- *Add vertex $moveload$ in gr_1.*
- *Add pre edge with label T=T,L1=L from vertex $moveload$ to vertex $attruck$ in gr_1.*
- *Add add edge with label T=T,L1=L from vertex $moveload$ to vertex $attruck$ in gr_1.*
- *Add del edge with label T=T,L1=L from vertex $moveload$ to vertex $attruck$ in gr_1.*
- *Map vertex v of gr_2 to vertex v of gr_1, where $v \in \{load, unload, move, attruck, atpackage, inpackage, incity, moveload\}$.*

6.2. Handling temporal domains

To handle negative preconditions, durative actions and action costs, full Algorithm 1 is needed. Graphs $\mathcal{G}_1$ and $\mathcal{G}_2$ in input are now ELDMGs, and Π'' is the ASP program reported in Listing 2.

The updates to steps (i)-(ii) to handle extended domains are in the following. The ASP program now operates on new atoms: val, $toMap$, and $mapValue$. These atoms have the following form:

- $val(cost\_or\_duration, id, status, graph, type, value, params)$ represents the costs and the durations of the durative actions in the graphs, where $cost\_or\_duration$ can be either $duration$ or $cost$, id identifies the duration or cost, $status$ denotes if the duration or the cost was in the input graph ($orig$) or if it must be added (add), $graph$ indicates the graph of the related durative action, $type$ indicates the type of duration or cost, and can be either $Function$ or $Natural\ Number$, $value$ represents the duration/cost when $type$ is equal to $Natural\ Number$ or the id of the function when $type$ is equal to $Function$, and $params$ denotes if the type of the cost/duration has been changed.

- $toMap(id1, id2)$ denotes the vertices, identified by a unique identifier $id1$ and $id2$, whose type t is equal to $(operator, durative)$ and that have been mapped by the encoding.

- $mapValue(id1, id2, params1, params2)$ denotes the mapping of the durations and costs of the two graphs, $\mathcal{G}_1$ and $\mathcal{G}_2$, respectively, identified by a unique identifier $id1$ and $id2$, while $params1$ and $params2$ denote the type of the durations and cost, which is $Function$ or $Natural\ Number$.

Further, rules of types 1 are updated to 1' as follows:

1'. $ver(id,orig,gr_i,name,t,"x_1,\ldots,x_k","x_1,\ldots,x_k")$.
for each vertex $name(x_1,\ldots,x_k)$ in $\mathcal{G}_i$(with $i \in \{1,2\}$) representing a vertex of type t, where t can be $(predicate, function)$ or $(operator, durative)$, and id is an identifier of the vertex.

and further rules connected to the new atoms are added:

5. $val(cost\_or\_duration, id,orig,gr_i,type,value,label)$.
for each vertex in $\mathcal{G}_i$(with $i \in \{1,2\}$) having t equals to $(operator, durative)$ representing the duration $cost\_or\_duration$ equal to $duration$ or the cost $cost\_or\_duration$ equal to $cost$ of the vertex with the identifier id, where $type$ is either $Natural\ Number$ or $Function$, and $value$ is a natural number if $type$ is equal to $Natural\ Number$, and $value$ is an id of a rule created in 1' if $type$ is equal to $Function$;
6. $\{val(cost\_or\_duration,id,add,gr_1,type,value,label)\}$.
for each vertex created with rules 3 with identifier id starting from a vertex in $\mathcal{G}_2$ with identifier id', such that there is an atom $val(cost\_or\_duration, id',orig,gr_2,type,value, label)$;
7. $\{val(cost\_or\_duration,id,orig,gr_1,type,value',label')\}$.
for each atom of the form $val(cost\_or\_duration, id,orig,gr_1,type,value,label)$ appearing in a rule created in 5, and for each atom of the form $val(cost\_or\_duration,id', orig,gr_2,type,value',label)$, with $value \neq value'$, and $label'$ is $"value, value'"$.

In step (ii), the ASP solver is executed (line 6 of Algorithm 1) on the resulting program now extended with $\Pi' \cup \Pi''$ (line 5): by exploiting the modularity of ASP, we were able to handle temporal domains by adding the following rules:

```
r12 toMap(ID1,ID2) :- map(ID1,ID2,_,_), ver(ID1,_,gr1,_,"type(Operator,Durative)",_,_).
r13 {mapValue(ID1,ID2,N1,N2) : val(ValueType,ID2,_,gr2,T,_,N1), val(ValueType,_,gr1,T,_,N2)} = 1
      :- toMap(ID1,ID2), val(ValueType,ID1,_,gr1,_,_,_).
r14 val(ValueType,ID1,orig,gr1,"Natural Number",V,"('Function';'Natural Number')") :-
      toMap(ID1,ID2), val(ValueType,ID2,_,gr2,"Natural Number",V,_),
      val(ValueType,ID1,_,gr1,"Function",_,_).
r15 :- mapValue(ID1,ID2,_,N), val(ValueType,ID2,_,gr2,"Natural Number",V,_), not
      val(ValueType,ID1,_,gr1,"Natural Number",V,N).
r16 :- mapValue(ID1,ID2,N,_), val(ValueType,ID2,_,gr2,"Function",V2,N),
      val(ValueType,ID1,_,gr1,"Function",V1,_), not map(V1,V2,_,_).
r17 :- val(ValueType,ID,_,gr2,_,_,_), #count{V : mapValue(V,ID,_,_)} != 1.
r18 :~ val(ValueType,ID,add,_,_,_,_). [1@2, ID]
r19 :~ val(ValueType,ID,_,gr1,T1,_,_), val(ValueType,ID,_,gr1,T2,_,_), T1 > T2. [1@2, ID]
r20 :~ val(ValueType,ID,_,gr1,T,V1,_), val(ValueType,ID,_,gr1,T,V2,_), V1 > V2. [1@2, ID, V1]
```

Listing 2: Extension of ASP Program Π'.

Rule r_{12} gets the pair of durative actions that have been mapped. Rule r_{13}, starting from the atoms inferred in r_{12}, creates a mapping between the duration and the costs of the graph $\mathcal{G}_1$

to exactly one duration and one cost of graph $\mathcal{G}_2$, respectively. Rule r_{14} generates an atom of type Natural Number for the durations and costs that are of type Function in the original graph but that should be mapped to a duration or cost of type Natural Number. Rules r_{15} and r_{16} ensure that the mapped durations and costs have the same values and that the relative functions are mapped, if the duration or cost are of type Natural Number and Function, respectively. Rule r_{17} ensures that each duration and cost of the graph $\mathcal{G}_2$ are mapped to exactly one duration and cost. Weak constraints r_{18} and r_{19} minimise the number of added durations and costs, and changes of type. Then, weak constraint r_{20}, with a lower priority level, minimises the changes of values.

Finally, as before, the behaviour of $ProcessSolution(A)$ at line 7 of Algorithm 1, which is detailed in Algorithm 2, is presented within a full execution of Algorithm 1 in the following Example 6.

Example 6 (Example 5 with durative actions)**.** *Let $\mathcal{G}_1$ and $\mathcal{G}_2$ be the ELDMGs of the Logistics domains considered in Example 5 but with the "macro" action being $durative$ and with a duration expressed by the same numeric fluent as in Example 3. Thus, $\mathcal{G}_2$ consists of the vertices of $\mathcal{G}_1$ extended with a vertex of the form $moveload(?P, ?T, ?L1, ?L2, ?C)$ having a duration represented by a numeric fluent. Since in this example there is a durative action involved, Algorithm 1 produces the same rules presented in Example 5 but for two rules of type 3: $\{ver($7, add, gr_1, $moveload$, $(operator, durative)$, "C, L, L, P, T", "C, L, L, P, T"$)\}$, since there is only one vertex in $\mathcal{G}_2$ of type $(operator, durative)$ that is not in $\mathcal{G}_1$ and one rule $\{ver($8, add, gr_1, $distance$, $(predicate, function)$, "C, L, L, P, T", "C, L, L, P, T"$)\}$. Moreover, it also adds a rule of type (6): $\{val($duration, 7, add, gr_1, $Function$, 8, "8;8"$)\}$.*

Then, Algorithm 1 analyses its output (from line 2 on) and produces the same instructions as in Example 5 but for the rules related to the durative action which become:

- *Add duration with id 7 of type $Function$ with value 8 in gr_1.*
- *Map value $distance$ of gr_1 to value $distance$ of gr_2.*

7. Experimental Analysis

Our experimental analysis aims to assess the ability of the proposed ASP-based approach to compare domain models of different sizes, and to generate optimally minimum number of changes that allow for transforming a model into the other one. Particularly, we are interested in understanding what dimensions of domain models affect the scalability of the proposed approach, and its ability to generate one or more optimally minimum number of changes accordingly.

Moreover, to demonstrate a practical exploitation of the approach in the knowledge engineering process, we present how our approach can be used to extract a common core of actions and predicates among different domains.

7.1. Benchmark models

To fully assess the capabilities of the proposed approach, in this analysis we consider both classical and temporal domain benchmarks. With regards to classical planning, we selected 7 different domain models from well-known international competitions. In particular, we considered the domains of Barman, Blocksworld, (simplified) Logistics, Rovers, Satellite,

and Sokoban from various editions of the International Planning Competition (IPC)[3], and the RPG and Match-3 domains from the ICKEPS 2016[4]. In the simplified Logistics, we consider only truck movements to obtain a more compact model. For temporal planning models, we selected 4 domain models from IPC 2018, maximising their diversity in terms of application domain and overall size. The selected domains are Cushing, Floortile, Parking, and Sokoban.

Models from the IPC are well-refined to support a wide range of planning engines in generating solutions to complex tasks, while ICKEPS models are designed with a focus on the knowledge engineering processes and on the quality of the models to be maintained and extended according to future needs (McCluskey et al., 2017; Chrpa et al., 2017). For these reasons, considering benchmarks from both competitions provides an ideal ground to assess the capabilities of the proposed method. This is also reflected in the technique we implemented to generate different models for the same domain. In all domains, both the classical and the temporal ones, but RPG and Match-3, we reformulated the original models by considering a mix of entanglements and macro-actions (Alarnaouti et al., 2023), and by modifying preconditions and effects of original operators. For the RPG and Match-3 domains, we compared two models crafted by two of the teams that took part in the competition: such models present significant differences in terms of predicates and operators as they embody very different interpretations of the domain at hand. In a nutshell, the RPG domain models a simplified role-playing game instance, where an agent has to navigate through a maze, that includes a range of rooms with various threats and obstacles. One approach, implemented by a team that took part in ICKEPS, is to accurately model all possible interactions between the agent and the environment (even those that are leading to the death of the agent), and leave to the planning engine the task of identifying safe actions and exploiting them. A radically different approach is to model only the interactions that are safe to be used by an agent, hence reducing the computational complexity for the planning engine. However, this comes at the cost of a decreased ability of the model to accurately replicate the encoded domain. While we are not advocating for one approach to be best, we reckon it is in the interest of the research community to be able to compare such models and to assess their differences, in order to access initial insights into how groups of experts may differ in developing models. The Match-3 domain is a classical planning benchmark modelled after popular tile-matching puzzle games. The problem involves a two-dimensional grid populated by various types of tiles. The actions are to swap two orthogonally adjacent tiles and to remove 3 matching tiles. The complexity comes from the fact that some combinations of swaps can destroy the involved tiles. The considered models differ in how they represent this tile-destroying dynamics – particularly in terms of supporting or preventing its use.

For the considered classical planning models, the number of operators ranges between 2 and 12, and the number of predicates between 4 and 25, while for the temporal planning models, the number of durative actions ranges between 3 and 6, and the number of predicates between 3 and 10. Details of the (E)LDMGs corresponding to the models are shown in Table 1, where the column Year indicates when the model was introduced. (Note that, in the experiments, it is no longer assumed $|\mathcal{G}_2| \geq |\mathcal{G}_1|$, since the implementation supports all cases.)

[3] https://www.icaps-conference.org/competitions/

[4] Encoding and benchmarks are available at: https://github.com/MarcoMochi/AIJ-planning-similarity.

Table 1: Overview of the considered benchmark models. Vertices and Edges give information about the size of the graphs to compare, $\mathcal{G}_1$ is the graph obtained by considering the original domain model, $\mathcal{G}_2$ is obtained from the reformulated model for all the domains but RPG, where two independently crafted original models are compared. RPG and Match-3 were presented at ICKEPS, while the other domains have been used as IPC benchmarks.

Domain	Year	Vertices		Edges	
		$\mathcal{G}_1$	$\mathcal{G}_2$	$\mathcal{G}_1$	$\mathcal{G}_2$
Sokoban	2008	6	7	16	18
Logistics	1998	7	8	13	21
Blocksworld	2000	9	11	27	29
Satellite	2002	13	16	23	44
Barman	2011	27	29	97	123
Rovers	2002	34	39	75	103
Sokoban	2018	9	7	36	16
Parking	2018	10	7	32	22
Floortile	2018	19	14	45	33
Cushing	2018	9	7	36	16
RPG	2016	13	34	23	74
Match-3	2016	14	12	58	57

7.2. Experimental Settings

We performed experiments on an Apple M1 CPU @ 3.22 GHz machine with 8 GB of physical RAM. Each system run was given an overall memory limit of 6 GB and 5 CPU-time minutes. As ASP system we used the state-of-the-art tool CLINGO (Gebser et al., 2016) 5.6.2, configured with the option `--parallel-mode=4`, which enables the use of multiple threads (with different solving strategies). We used 4 threads, and in our experiments this helps improve the performance of CLINGO compared to the default configuration. Moreover, we used the open-source python library PYSPEL (Alviano et al., 2021) which simplifies the implementation of Algorithm 1. We tested two approaches according to the value of the $Preprocessing$ variable.

7.3. Results

Firstly, we tested on all the original domains, the ability of our solution to compare models that are exactly the same but for the names of the involved operators and predicates, i.e., if they are strongly equivalent. The results, as presented in Table 2, indicate that the ASP solution employing preprocessing always identifies the compared graphs as isomorphic, and provides an appropriate mapping between the nodes of the compared LDMGs, in less than 0.1 CPU-time seconds while, without using the preprocessing technique, the solution is not able to handle the Barman and Rovers domains because they exceed the memory limit. The better scalability obtained with the usage of preprocessing is highlighted by the big increase in the number of rules generated by the solutions without preprocessing in different domains compared to the solutions that use preprocessing.

Then, we move to the general case and the results of the experimental analysis are shown in Table 3 where, for each domain and tested approach, we report the number of optimal models found, the minimum number of elementary operations needed to make the models strongly

Table 2: Time required to find the optimal solution in the generated graphs for the strong equivalence case. Results are presented in terms of seconds needed to find the Optimal solution. (# Rules) shows the size of the ASP program.

Domain	*Preprocessing*=false		*Preprocessing*=true	
	CPU Time	# Rules	CPU Time	# Rules
Sokoban	0.1	1,431	0.1	652
Logistics	0.1	3,235	0.1	791
Blocksworld	0.1	163,256	0.1	58,687
Satellite	0.1	137,684	0.1	3,353
Barman	–	–	0.1	43,432
Rovers	–	–	0.1	46,844
RPG	0.1	13,801	0.1	8,022
Match-3	0.1	29,859	0.1	5,035

equivalent, the CPU time to find the optimal solutions, and the number of rules generated by the grounding. The table is divided horizontally into three parts: The top part considers cases where a model has been reformulated, the middle part is the ICKEPS domains, while the bottom part focuses on comparing very different models (Barman vs Logistics, Rovers vs Satellite) as a stress test for our approach. Square brackets indicate that an optimal solution has been found in the reported CPU time (checked manually), but has not been proved by CLINGO. As a first observation, the approach employing preprocessing is extremely fast, since for all the tested benchmarks we are able to find an optimal solution (though not always proved) in less than 1 CPU-time second. Instead, plain ASP encoding exceeds the memory limits when executed on large domains (Barman and Rovers) or in the presence of significant differences between the compared domains, showing that preprocessing is indeed necessary in challenging cases. Note that even the mid-size Satellite domain leads to more than half a million rules, whereas the approach using preprocessing produces only around five thousand rules. In models with few tens of edges, there is no substantial difference between finding one optimal solution and finding all of them. This result can be explained by the fact that the number of optimal solutions is rather small (maximum 4) and this paves the way for the development of more comparison techniques, e.g., by proposing preferences among the possible solutions.

Turning our attention to the analysis in the bottom part of the table, which is expected to be challenging, it is easy to notice that despite the fact that for the RPG domain required 4 additional nodes and 78 edges, removals and remapping of the parameters, the proposed solution was able to generate an optimal result for this case and for the comparison of different domains in less than 1 CPU-time second. However, differently from the other tests, in the cases of different domains the approach was able to generate a single optimal solution and not to enumerate all the optimal ones.

After the evaluation of our approach on classical planning models, we further tested it with temporal planning models, following a similar process. We began by checking, on all the original domains, if our solution was able to detect strong equivalence between models. As can be seen in Table 4, which is organized as Table 2, even with durative actions, our solution can verify strong equivalence between two models almost immediately, both not using

Table 3: Performance of the proposed ASP-based approach on classical domains without/with preprocessing. Results are presented in terms of seconds needed to enumerate all the optimal solutions, and the number of optimal models (Opt Mod.). Square brackets indicate that the optimal solution was not proved. (# of Changes) represents the minimum number of elementary operations needed. (# Rules) shows the size of the ASP program.

Domain	Opt. Mod.	# Of Changes	*Preprocessing*=false CPU Time	*Preprocessing*=false # Rules	*Preprocessing*=true CPU Time	*Preprocessing*=true # Rules
Sokoban	1	3	0.1	1,975	0.1	636
Logistics	1	11	0.1	7,106	0.1	697
Blocksworld	2	37	0.1	3,324	0.1	1,545
Satellite	2	30	2.7	536,020	0.1	5,476
Barman	1	32	–	–	0.3	50,210
Rovers	4	38	–	–	0.4	63,522
RPG	[1]	113	[1.9]	338,676	[0.4]	16,522
Match-3	[1]	89	[1.5]	62,121	[0.5]	9,467
Barman_Log.	[1]	34	–	–	[0.1]	39,088
Rovers_Sat.	[1]	33	–	–	[0.8]	16,565

Table 4: Time required to find the optimal solution in the generated graphs for the strong equivalence of temporal domains, all containing durative actions. Results are presented in terms of seconds needed to find all optimal solutions. (# Rules) shows the size of the ASP program.

Domain	*Preprocessing*=false CPU Time	*Preprocessing*=false # Rules	*Preprocessing*=true CPU Time	*Preprocessing*=true # Rules
Sokoban	0.1	18,989	0.1	1,947
Parking	0.1	20,492	0.1	5,277
Floortile	1.2	347,333	0.1	10,679
Cushing	0.1	7,864	0.1	1,506

and using the preprocessing technique. Then, we tested each domain against a slightly modified version of itself. In the top half of Table 5, which is organized as Table 3, are presented the results obtained in each of the domains. As in the previously presented experiments, the ASP-based approach can find an optimal solution in less than 0.1 CPU-time seconds when employing preprocessing. Moreover, within this time frame, it is able to enumerate all optimal solutions, which can now also be a consistent number. Without the preprocessing, the number of generated rules is significantly larger, and, as expected, this deteriorates the solution's performance. Next, in the bottom part of Table 5, we conducted more demanding tests by comparing different domains. In this way, each domain is compared to a very different alternative. Without preprocessing, the solution is able to prove the optimality of only one problem. In the other cases, the solution is able to find an optimal solution in a few seconds, but not to prove it within the time limit. Moreover, in these instances, the advantages of using the preprocessing step are even larger in terms of the number of generated rules. Indeed, in two instances, more than 1 million rules are generated without preprocessing compared

Table 5: Performance of the proposed ASP-based approach with preprocessing on temporal domains, all containing durative actions. Results are presented in terms of seconds needed to enumerate all the optimal solutions, and the number of optimal models (Opt Mod.). Square brackets indicate that the (last) optimal solution was not proved. (# of Changes) represents the minimum number of elementary operations needed for extended domains. (# Rules) shows the size of the ASP program.

Domain	Opt. Mod.	# Of Changes	*Preprocessing*=false CPU Time	*Preprocessing*=false # Rules	*Preprocessing*=true CPU Time	*Preprocessing*=true # Rules
Sokoban	256	30	0.1	11,505	0.1	2,123
Parking	12	27	0.3	10,927	0.1	1,865
Floortile	2	33	[67.2]	150,227	0.1	8,136
Cushing	720	19	0.1	12,756	0.1	1,374
Sokoban_Parking	[1]	115	[7.3]	109,093	0.3	4,295
Sokoban_Floortile	[1]	124	[28.4]	1,995,031	0.5	7,732
Sokoban_Cushing	[1]	75	17.3	10,906	0.1	2,723
Parking_Floortile	[1]	122	[10.4]	1,389,805	1.1	6,687
Parking_Cushing	[1]	77	[0.6]	9,663	0.4	5,910
Cushing_Floortile	[1]	108	[2.3]	36,227	2.2	16,076

to less than 10 thousand rules when using preprocessing. As previously noted, when the domains are similar, the enumeration of all the solutions is completed quickly. When comparing different domains, while one solution is still found rapidly, the enumeration of all the optimal solutions is challenging.

Summarising, the performed experimental analysis indicates that the presented ASP solution, enhanced with the preprocessing step, is very efficient in generating optimally minimum number of modifications, even when comparing one optimal solution of different models that allows to transform a model into the compared one, on the basis of the corresponding (E)LDMGs. Considering that results are generated efficiently, the proposed tool can also be exploited during the domain encoding step for comparing alternative representations of a domain's dynamics.

7.4. Common core

Finally, we are interested in assessing how the proposed approach can provide an operational tool for the knowledge engineering process of planning domain models. One of the potential uses of the concept of similarity in the context of knowledge engineering is the comparison of different planning domain models to identify common structures: a common set of actions or predicates between the models that interact with the other actions and predicates similarly. In this way, it is possible to identify a set of actions and predicates in each model, the core, that can express a similar behaviour shared among the models. For instance, it can be possible to clearly identify how the movement of vehicles has been encoded in PDDL benchmarks, or how loading and unloading operations have been modelled. The core is then the ideal candidate to become a design pattern for planning models, or to support notions such as inheritance and polymorphism, that knowledge engineers for planning borrowed from the more traditional field of software engineering (Lindsay, 2023; Vallati and McCluskey, 2020).

The problem of extracting the core from a set of models can be addressed using our ASP-based approach to compare domain models. It can be achieved by deriving the mapping between the models each pair at a time. Then, having evaluated the mapping of each possible pair of models, it is possible to derive the core by deriving the chain of actions and predicates mapped together in all the found mappings. In the following, we present the results of an analysis obtained by employing Algorithm 1 using 3 different planning models that work on declinations of the logistics problem: Logistics, Driverlog, and Depots, taken from the IPC benchmarks.

To extract the core, we obtained the mapping of actions and predicates between all pairs of models: Logistics and Driverlog, Logistics and Depots, and finally Driverlog and Depots. To identify the core, we checked whether the actions and predicates in the Driverlog to which the actions and predicates of the Logistics model are mapped are also mapped to each other in the comparison between the Driverlog and Depots models.

The actions in the different domains are the following:

- Logistics: $load$, $unload$, $move$
- Driverlog: $load\text{-}truck$, $unload\text{-}truck$, $board\text{-}truck$, $disembark\text{-}truck$, $drive\text{-}truck$
- Depots: $walk$, $drive$, $lift$, $drop$, $load$, $unload$

Comparing the Logistics model with the other models, we obtained that the action $move$ is mapped to $drive\text{-}truck$ of the Driverlog model and to $drive$ of the Depots model. Thus, to define these actions as part of the core, we should check if $drive$ of the Depots domain and $drive\text{-}truck$ of the Driverlog domain are mapped when comparing Driverlog and Depots. In this case, we found that there is such a mapping and thus it is possible to state that these three actions are part of the common core. Similarly, we checked all the other actions and found that a common core is composed of the following actions (for each item of the list, the first action belongs to the Logistics domain, the second one to the Driverlog domain and the last one to the Depots domain) :

- $move$, $drive\text{-}truck$, $drive$
- $load$, $load\text{-}truck$, $load$
- $unload$, $unload\text{-}truck$, $unload$

Thus, it is possible to affirm that all the actions in the Logistics model compose a common core of the three planning models and that these models, through the actions in the core, share a common behaviour. In knowledge engineering terms, this can suggest that to represent the basic dynamics of moving vehicles that can be loaded, the identified set of actions and predicates provide the ideal building ground on which new models can be based. While the considered case can be seen as trivial, it is worth noting that this is the first approach that allows to identify, in an automated way, shared structures between PDDL planning models.

8. Conclusion

This paper contributes to the theory and practice of the problem of comparing planning domain models, by defining the domain model similarity problem, which builds on the notion of strong equivalence of domain models, also introduced in the paper. We proposed an

approach based on Answer Set Programming for specifying and solving the problem – with particular attention given to the identification of optimal minimum number of modifications that allow to transform one model into the compared one. Experiments on benchmarks from classical and temporal domains of different sizes show that the approach can solve the domain model similarity problem in a very short time when preprocessing is enabled. We also applied our solution for solving the problem of computing "common cores" of domain models that can identify common parts of multiple domain models in order to determine their "class-like" hierarchy that allows for leveraging the novel concept of action inheritance (Lindsay, 2023).

Future work will focus on testing our approach on the application domains (i)-(iii) mentioned in the introduction, and on its extension to more expressive planning representation languages, such as PDDL+ (Fox and Long, 2006), that can also consider hybrid discrete-continuous numeric changes, and improve the explanations provided as output. Furthermore, we plan to investigate how different changes to the model affect its expressivity and, possibly, the sets of plans it can admit.

Acknowledgements

L. Chrpa was funded by the Czech Science Foundation (project no. 25-18003S) and by the European Union under the project ROBOPROX (reg. no. CZ.02.01.01/00/22_008/0004590). M. Vallati was supported by the UKRI Future Leaders Fellowship [grant number MR/T041196/1]. C. Dodaro was supported by Italian Ministry of Research (MUR) under PNRR projects FAIR "Future AI Research", CUP H23C22000860006, and Tech4You "Technologies for climate change adaptation and quality of life improvement", CUP H23C22000370006;

Declaration of competing interest

The authors declare none.

Data availability

A link containing the code and the benchmarks employed in the paper is included in the article.

References

L. Chrpa, C. Dodaro, M. Maratea, M. Mochi, M. Vallati, Comparing planning domain models using answer set programming, in: Proc. of JELIA, 2023, pp. 227–242.

M. Ramírez, M. Papasimeon, N. Lipovetzky, L. Benke, T. Miller, A. R. Pearce, E. Scala, M. Zamani, Integrated hybrid planning and programmed control for real time UAV maneuvering, in: Proc. of AAMAS, 2018, pp. 1318–1326.

M. Ai-Chang, J. Bresina, L. Charest, A. Chase, J.-J. Hsu, A. Jonsson, B. Kanefsky, P. Morris, K. Rajan, J. Yglesias, et al., Mapgen: mixed-initiative planning and scheduling for the mars exploration rover mission, IEEE Intelligent Systems 19 (2004) 8–12.

M. Cardellini, M. Maratea, M. Vallati, G. Boleto, L. Oneto, In-station train dispatching: a PDDL+ planning approach, in: Proc. of ICAPS, 2021, pp. 450–458.

T. L. McCluskey, J. M. Porteous, Engineering and compiling planning domain models to promote validity and efficiency, Artificial Intelligence 95 (1997) 1–65.

T. L. McCluskey, T. S. Vaquero, M. Vallati, Engineering knowledge for automated planning: Towards a notion of quality, in: Proc. of K-CAP, 2017, pp. 14:1–14:8.

M. Vallati, T. L. McCluskey, A quality framework for automated planning knowledge models, in: Proc. of ICAART, 2021, pp. 635–644.

M. Vallati, L. Chrpa, On the robustness of domain-independent planning engines: The impact of poorly-engineered knowledge, in: Proc. of K-CAP, 2019, pp. 197–204.

J. Oswald, K. Srinivas, H. Kokel, J. Lee, M. Katz, S. Sohrabi, Large language models as planning domain generators, in: Proc. of ICAPS, 2024, pp. 423–431.

A. Shrinah, D. Long, K. Eder, D-VAL: An automatic functional equivalence validation tool for planning domain models, arXiv preprint arXiv:2104.14602 (2021).

A. Coulter, T. Ilie, R. Tibando, C. Muise, Theory alignment via a classical encoding of regular bisimulation, in: KEPS Workshop, 2022.

T. Chakraborti, S. Sreedharan, Y. Zhang, S. Kambhampati, Plan explanations as model reconciliation: Moving beyond explanation as soliloquy, in: Proc. od IJCAI, 2017, pp. 156–163.

S. Shoeeb, T. McCluskey, On comparing planning domain models, in: PlanSIG Workshop, 2011.

M. Fox, D. Long, PDDL2.1: an extension to PDDL for expressing temporal planning domains, Journal of Artificial Intelligence Research 20 (2003) 61–124.

C. Baral, Knowledge Representation, Reasoning and Declarative Problem Solving, Cambridge University Press, 2003.

M. Gelfond, V. Lifschitz, Classical Negation in Logic Programs and Disjunctive Databases, New Generation Computing 9 (1991) 365–386.

I. Niemelä, Logic Programs with Stable Model Semantics as a Constraint Programming Paradigm, Annals of Mathematics and Artificial Intelligence 25 (1999) 241–273.

G. Brewka, T. Eiter, M. Truszczynski, Answer set programming at a glance, Communications of the ACM 54 (2011) 92–103.

S. Cresswell, T. L. McCluskey, M. M. West, Acquiring planning domain models using *LOCM*, The Knowledge Engineering Review 28 (2013) 195–213.

L. Chrpa, T. L. McCluskey, M. Vallati, T. Vaquero, The fifth international competition on knowledge engineering for planning and scheduling: Summary and trends, AI Magazine 38 (2017) 104–106.

A. Lindsay, On using action inheritance and modularity in pddl domain modelling, in: Proc. of ICAPS, 2023, pp. 259–267.

S. Biundo, R. Aylett, M. Beetz, D. Borrajo, A. Cesta, T. Grant, T. McCluskey, A. Milani, G. Verfaillie, PLANET roadmap, http://www. planet-noe.org/service/Resources/Roadmap/Roadmap2.pdf (2003).

S. Aitken, N. R. Shadbolt, Knowledge level planning, in: Current Trends in AI Planning-Proc. of EWSP, 1994.

M. M. S. Shah, L. Chrpa, D. E. Kitchin, T. L. McCluskey, M. Vallati, Exploring knowledge engineering strategies in designing and modelling a road traffic accident management domain, in: Proc. of IJCAI, 2013, pp. 2373–2379.

R. Simpson, T. Mccluskey, D. Long, M. Fox, Generic types as design patterns for planning domain specification, in: KETTAIP Workshop, 2002.

M. Vallati, T. L. McCluskey, In defence of design patterns for AI planning knowledge models, in: Proc. of AIxIA, 2020, pp. 191–203.

R. M. Simpson, D. E. Kitchin, T. L. McCluskey, Planning domain definition using gipo, The Knowledge Engineering Review 22 (2007) 117–134.

T. S. Vaquero, V. Romero, F. Tonidandel, J. R. Silva, itsimple 2.0: An integrated tool for designing planning domains, in: Proc. of ICAPS, 2007, pp. 336–343.

G. Wickler, L. Chrpa, T. L. McCluskey, Kewi: A knowledge engineering tool for modelling ai planning tasks, in: Proc. of KEOD, 2014, pp. 36–47.

K. Mourão, L. Zettlemoyer, R. P. Petrick, M. Steedman, Learning strips operators from noisy and incomplete observations, in: Proc. of UAI, 2012, pp. 614–623.

P. Gregory, A. Lindsay, Domain model acquisition in domains with action costs, in: Proc. of ICAPS, 2016, pp. 149–157.

A. Arora, H. Fiorino, D. Pellier, M. Métivier, S. Pesty, A review of learning planning action models, The Knowledge Engineering Review 33 (2018) e20.

A. Mordoch, E. Scala, R. Stern, B. Juba, Safe learning of PDDL domains with conditional effects, in: Proc. of ICAPS, 2024, pp. 387–395.

D. Aineto, E. Scala, Action model learning with guarantees, in: Proc. of KR, 2024.

A. Lindsay, S. Franco, R. Reba, T. L. McCluskey, Refining process descriptions from execution data in hybrid planning domain models, in: Proc. of ICAPS, 2020, pp. 469–477.

J. Porteous, J. F. Ferreira, A. Lindsay, M. Cavazza, Automated narrative planning model extension, Autonomous Agents and Multi-Agent Systems 35 (2021) 19.

L. Chrpa, M. Vallati, Planning with critical section macros: theory and practice, Journal of Artificial Intelligence Research 74 (2022) 691–732.

D. Alarnaouti, G. Baryannis, M. Vallati, Reformulation techniques for automated planning: a systematic review, The Knowledge Engineering Review 38 (2023) e9.

S. Sreedharan, T. Chakraborti, S. Kambhampati, Foundations of explanations as model reconciliation, Artificial Intelligence 301 (2021) 103558.

S. Sreedharan, A. O. Hernandez, A. P. Mishra, S. Kambhampati, Model-free model reconciliation, in: Proc. of IJCAI, 2019, pp. 587–594.

D. Aineto, S. Jiménez, E. Onaindia, M. Ramírez, Model recognition as planning, in: Proc. of ICAPS, 2019, pp. 13–21.

C. Grundke, G. Röger, M. Helmert, Formal representations of classical planning domains, in: Proc. of ICAPS, 2024, pp. 239–248.

V. Nguyen, V. L. Stylianos, T. C. Son, W. Yeoh, Explainable planning using answer set programming, in: Proc. of KR, 2020, pp. 662–666.

T. C. Son, V. Nguyen, S. L. Vasileiou, W. Yeoh, Model reconciliation in logic programs, in: Proc. of JELIA, 2021, pp. 393–406.

V. Nguyen, T. C. Son, W. Yeoh, Explainable problem in clingo-dl programs, in: Proc. of SoCS, 2021, pp. 231–232.

T. Janhunen, R. Kaminski, M. Ostrowski, S. Schellhorn, P. Wanko, T. Schaub, Clingo goes linear constraints over reals and integers, Theory and Practice of Logic Programming 17 (2017) 872–888.

M. Ghallab, D. Nau, P. Traverso, Automated Planning: Theory & Practice, Morgan Kaufmann, 2004.

D. Mcdermott, M. Ghallab, A. Howe, C. Knoblock, A. Ram, M. Veloso, D. Weld, D. Wilkins, PDDL - The Planning Domain Definition Language, Technical Report, CVC TR-98-003/DCS TR-1165, Yale Center for Computational Vision and Control, 1998.

R. Fikes, N. J. Nilsson, STRIPS: A new approach to the application of theorem proving to problem solving, Artificial Intelligence 2 (1971) 189–208.

E. Bengoetxea, Inexact Graph Matching Using Estimation of Distribution Algorithms, Ph.D. thesis, Ecole Nationale Supérieure des Télécommunications, Paris, France, 2002.

A. Sanfeliu, K. Fu, A distance measure between attributed relational graphs for pattern recognition, IEEE Transactions on Systems, Man, and Cybernetics 13 (1983) 353–362.

F. Calimeri, W. Faber, M. Gebser, G. Ianni, R. Kaminski, T. Krennwallner, N. Leone, M. Maratea, F. Ricca, T. Schaub, ASP-Core-2 input language format, Theory and Practice of Logic Programming 20 (2020) 294–309.

W. Faber, G. Pfeifer, N. Leone, Semantics and complexity of recursive aggregates in answer set programming, Artificial Intelligence 175 (2011) 278–298.

F. Buccafurri, N. Leone, P. Rullo, Enhancing disjunctive datalog by constraints, IEEE Transactions on Knowledge and Data Engineering 12 (2000) 845–860.

J. Köbler, U. Schöning, J. Torán, The Graph Isomorphism Problem: Its Structural Complexity, Progress in Theoretical Computer Science, Birkhäuser/Springer, 1993.

Z. Zeng, A. K. H. Tung, J. Wang, J. Feng, L. Zhou, Comparing stars: On approximating graph edit distance, Proc. of VLDB Endow. 2 (2009) 25–36.

M. Gebser, R. Kaminski, B. Kaufmann, M. Ostrowski, T. Schaub, P. Wanko, Theory solving made easy with clingo 5, in: Proc. of ICLP (Technical Communications), 2016, pp. 2:1–2:15.

M. Alviano, C. Dodaro, A. Previti, Python Specification Language, https://github.com/dodaro/pyspel, 2021.

M. Fox, D. Long, Modelling Mixed Discrete-Continuous Domains for Planning, Journal of Artificial Intelligence Research 27 (2006) 235–297.